Affine Runtime Calibration for Drift-Adaptive Gottesman–Kitaev–Preskill Decoding

Anonymous authors

Abstract

Approximate Gottesman–Kitaev–Preskill (GKP) error correction converts continuous modular syndrome measurements into displacement corrections. This conversion becomes difficult when finite-energy states, noisy measurement chains and drifting operating conditions move the syndrome distribution over time. Fully learned per-shot decoders can be hard to bound in a real-time control stack, whereas fixed affine correction laws are fast but brittle. We therefore formulate drift-adaptive GKP decoding as affine runtime calibration: the latency-critical path is restricted to applying $\Delta_t = K_t s_t + b_t$, and a slower loop updates (K_t, b_t) from recent analog syndrome statistics.

This formulation targets a gap between two mature lines of work. GKP and bosonic-code decoders increasingly exploit analog reliability information, but the updated object is often an outer-code weight, a message-passing likelihood or a full decoder state. Real-time hardware decoders, in contrast, succeed by making the online datapath small, clockable and reproducible. Our contribution is to connect these requirements at the physical correction layer: recent analog statistics are used to recalibrate a low-dimensional affine displacement surface, while each shot still sees a deterministic matrix-vector rule.

In a predeclared four-scenario, two-repeat software-in-the-loop simulation benchmark, the main-table hybrid residual branch has the lowest descriptive mean `final_ler_mean` in every tested drift scenario. The scenario-wise mean reduction relative to the unscented Kalman filter is 1.8–2.9%, with a 2.3% average across the tested scenarios; these values are descriptive, not inferential. Ablations and a six-seed mechanism analysis indicate that histogram dynamics and teacher-side features affect the result, but they do not prove a unique causal mechanism or teacher necessity. A supplementary statistical-calibration analysis further shows that a non-neural estimator can populate the same affine interface, supporting the contract rather than a specific neural module. Controlled oracle and wrapped-Gaussian checks separate estimator error from local affine-model limits, and an analytical operation-count model shows why the affine rule is a low-complexity fast-path candidate. Board timing, resource use, power and source-vs-board measurements are not reported in this study. The evidence therefore supports a simulation-first systems-method claim: low-dimensional affine calibration is a plausible organizing principle for drift-adaptive GKP fast-path decoding, while deployment-level and broad-benchmark claims require additional validation.

1 Introduction

Bosonic quantum error correction encodes information in oscillator degrees of freedom and can exploit continuous measurement records that are unavailable in binary stabilizer readout. The GKP code is a prominent example: shift errors are inferred from modular quadrature syndromes and corrected by displacement operations [**?**, **?**]. In an idealized setting, the decoder can treat each measured syndrome as a local displacement estimate. In a physical setting, however, approximate finite-energy GKP states, noisy auxiliary measurements and hardware calibration drift alter the distribution seen by the decoder over time.

The resulting bottleneck is not only statistical accuracy, but the interface between estimation and real-time control. A drift-aware decoder should exploit recent analog syndrome information, yet a decoder intended for a hardware control stack must expose a small, testable and latency-bounded correction surface. End-to-end neural decoding can be expressive, but it obscures worst-case timing, fixed-point degradation and rollback behaviour. A fixed affine correction law has the opposite profile: it is fast and interpretable, but it cannot by itself follow bias, variance or orientation drift in the effective GKP noise state.

This manuscript separates these two responsibilities. The fast loop is limited to a two-quadrature affine rule,

$$\Delta_t = K_t s_t + b_t, \tag{1}$$

where s_t is the measured syndrome and (K_t, b_t) are the active runtime parameters. The slow loop updates these parameters from a recent syndrome histogram, teacher estimate or statistical calibration rule. The learned component is therefore not the per-shot decoder; it is one possible slow-loop module for recalibrating a low-dimensional physical displacement rule. Different estimators can be compared only after they commit the same type of (K, b) parameters to the deterministic fast path.

We test whether a slow calibration module can improve a deterministic affine GKP fast path under controlled drift while preserving a small, verifiable runtime interface. Under a predeclared software-in-the-loop simulation protocol, the main-table hybrid residual branch lowers the protocol-defined `final_ler_mean` in every tested drift scenario relative to the strongest classical runner-up. The present evidence is simulation-based and leaves board-level timing, resource use and source-vs-board agreement to future validation.

The comparisons in this paper are therefore tiered. The direct numerical comparison is the protocol-defined `final_ler_mean` ranking within the controlled drift benchmark. The fidelity discussion is a residual-boundary surrogate that explains how half-lattice crossings relate to channel language, not a finite-energy logical-channel reconstruction. The latency and resource discussion is a datapath argument based on operation count, fixed-point software parity and explicit hardware measurement requirements, not a measured FPGA result. This separation is central to the manuscript: it lets the reader distinguish the current simulation result from the channel-level and device-level standards used in adjacent work.

This tiering also defines the specific advantage being tested. The paper does not argue that an affine rule is a universal GKP decoder, nor that a small CNN should replace established soft-information or learned decoders. It argues that a slow calibration loop can absorb nonstationary syndrome statistics and commit a compact (K, b) surface whose online correction is cheaper to bound, quantize, log and eventually reproduce on hardware than a per-shot neural or multi-branch posterior calculation. The main evidence therefore concerns adaptation under drift and contract simplicity, not end-to-end fault-tolerance thresholds.

The new element is the interface between GKP analog statistics and a bounded runtime surface. First, the updated object is the physical-layer affine correction (K, b), not an outer-code graph weight or a full neural decision rule. Second, the slow-loop estimator can be neural, statistical or teacher anchored, provided that it commits the same low-dimensional surface to the fast path. Third, the evidence is organized around that shared contract: the main simulation ranking tests whether residual calibration improves `final_ler_mean`, while the cost, fixed-point and source-data analyses test whether the resulting fast path is a plausible hardware verification target.

The main contributions are:

1. A runtime-consistent formulation of drift-adaptive GKP decoding as affine calibration of (K, b), with the per-shot correction constrained to Eq. (1).

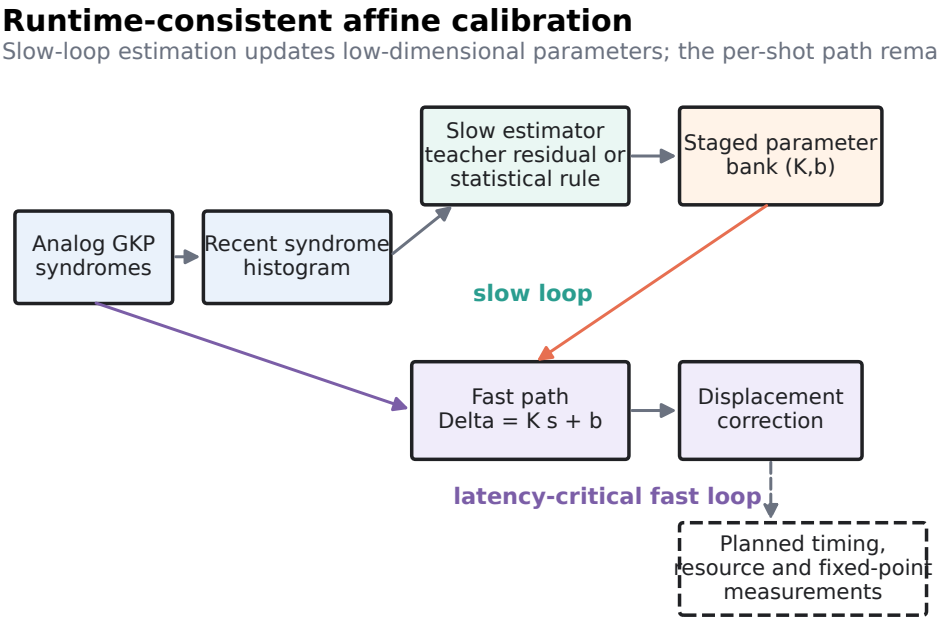

Figure 1: Overview of runtime-consistent affine calibration. The slow calibration loop updates low-dimensional (K, b) parameters from recent syndrome statistics, while the latency-critical fast path remains the deterministic affine operation in Eq. (1). The dashed hardware-facing surface marks future timing, fixed-point and resource measurements, not current hardware results.

2. A residual branch, instantiated here with teacher anchoring in the main comparison, that predicts a bounded bias calibration term while leaving the fast path deterministic.

3. A predeclared four-scenario software-in-the-loop simulation evaluation showing that the hybrid residual branch ranks lowest by mean `final_ler_mean` in all tested scenarios under the current two-repeat protocol.

4. A metric and validation ladder separating protocol-defined `final_ler_mean`, residual-boundary channel language, analytical fast-path cost, fixed-point software parity and the hardware measurements needed before deployment-level claims.

5. A comparison framework that positions the method against analog GKP decoding, adaptive-prior calibration, learned QEC modules and real-time FPGA decoders without treating incompatible metrics as a single leaderboard.

2 Related Work

GKP analog information. Analog GKP syndrome values carry reliability information that can improve decoding beyond hard thresholding. Prior work has used analog information in high-threshold GKP error correction, surface-GKP decoding and bosonic-QLDPC-GKP settings [?, ?, ?, ?, ?, ?]. The present work does not claim novelty in using analog information. Its distinction is the layer at which the information is used: recent analog syndrome statistics update a physical-layer affine displacement rule, rather than only an outer-code matching weight or message. The

Topic	Prior scope	Distinction of this work
GKP analog decoding	Uses continuous syndrome values or reliability weights for GKP or outer-code decoding.	Uses recent analog histograms to calibrate the physical-layer affine displacement rule itself.
Adaptive priors	Updates noise weights, graph weights or decoder priors from syndrome statistics.	Maps the estimated state to committed (K, b) parameters consumed by a deterministic fast path.
Learned QEC modules	Places neural modules before, inside or after a classical decoder.	Keeps the neural branch in the slow calibration loop rather than on the per-shot correction path.
FPGA/real-time decoders	Reports deployed low-latency or hardware-integrated decoders in stabilizer-code settings.	Provides a software-tested GKP physical-layer calibration contract; hardware results require future board-level measurements.

Table 1: Topic-level positioning. The contribution is the runtime-bounded affine calibration contract for a drifting GKP physical correction layer, not a dismissal of prior analog, learned or hardware-decoder work.

histogram window used here is also not a Bayesian multi-round approximate GKP memory decoder and does not model noisy auxiliary-state correlations [**?**, **?**]. It is an effective-noise calibration signal for a committed physical affine surface.

Adaptive priors and calibration-aware decoding. Decoder performance depends on the match between assumed and actual noise. Adaptive weight estimation, syndrome-statistics estimation and calibrated decoders address this mismatch by updating priors or graph weights from observed data [**?**, **?**, **?**, **?**, **?**]. Our formulation follows the same motivation, but constrains the updated object to the GKP affine fast-path parameters and commits those parameters through a stage-and-commit runtime contract.

Learned low-latency QEC modules. Machine-learning-assisted decoders and pre-decoders can reduce latency or improve accuracy when their role in a classical decoding stack is explicit [**?**, **?**, **?**, **?**, **?**, **?**]. Structured neural decoders, soft-information decoders and calibration-driven control updates have also been studied in bosonic, surface-code and hardware-data settings [**?**, **?**, **?**, **?**]. The residual module studied here is narrower than a full neural decoder: it predicts a bounded calibration term for a low-dimensional affine surface and is kept outside the per-shot critical path. The novelty claim is therefore not "neural GKP decoding", but the restriction of adaptation to a committed affine fast-path surface.

Real-time and FPGA QEC decoders. Hardware QEC decoders for stabilizer-code settings include lookup-table, union-find, matching-accelerator, clustering, greedy and FPGA-tailored message-passing systems [**?**, **?**, **?**, **?**, **?**, **?**, **?**, **?**]. These works define the standard for deployment claims: closed-loop latency, resource use, timing closure and hardware integration. Our present evidence does not meet that standard. Instead, this paper treats the GKP affine fast-path contract as the object to be tested in software first, with hardware-facing metrics reserved for future board-level measurements.

The comparisons below are metric-oriented rather than leaderboard-style. Prior work reports different code families, validation layers and endpoint metrics, so the tables identify the measurement standard each line establishes and the narrower metric reported here.

Taken together, the comparison defines a narrow advantage claim. The paper does not compete with surface-GKP or QLDPC-GKP decoders on end-to-end overhead, and it does not compete with real-time FPGA decoders on measured device latency. Its current advantage is architectural: the adaptive part of the decoder changes only a low-dimensional affine surface, leaving the per-shot path as a clipped matrix-vector rule. This gives the manuscript a testable route to three later hardware-facing metrics: lower analytical per-shot arithmetic count than the branch-aware posterior references evaluated in the controlled cost model, simpler verification than per-shot neural decoding and a clear source-vs-board agreement target for an FPGA implementation.

For clarity, the manuscript uses a metric ladder rather than a single all-purpose performance number. The primary result is the protocol-defined `final_ler_mean` ranking and paired descriptive delta. The residual-boundary surrogate is only a bridge from half-lattice crossing events to channel language, not a finite-energy logical-channel fidelity. The fast-path latency argument is currently an analytical and software-emulation argument, not an FPGA timing result. Table 4 separates the metrics reported in this study from the metrics that define the standard for later validation.

This ladder is the intended reading guide for all cross-paper comparisons in the manuscript.

3 Problem Setting

3.1 Approximate GKP syndrome model

The ideal square-lattice GKP code represents logical states as combs in phase space. In the position quadrature, $|0_L\rangle$ and $|1_L\rangle$ occupy alternating lattice points separated by $\sqrt{\pi}$, while stabilizer periods are $2\sqrt{\pi}$. Approximate GKP states replace the Dirac comb by finitely squeezed peaks with a finite-energy envelope, so both state preparation and readout add continuous noise to the modular syndrome. For example, one may view an approximate position-basis codeword schematically as a comb of Gaussian peaks with width Δ, multiplied by a broad envelope that limits the oscillator energy. The exact envelope and circuit depend on the physical preparation route, but the decoder-facing consequence is common: finite squeezing turns a sharp lattice-cell decision into a tail probability near the half-lattice boundary. A small change in residual scale or bias can therefore produce a disproportionately large change in boundary crossing probability when the residual distribution sits near that decision surface.

Let the accumulated two-quadrature displacement before correction be $e_t = (e_{q,t}, e_{p,t})^\top$. The implementation uses the lattice period $\lambda = \sqrt{2\pi}$ for syndrome wrapping and half-lattice residual boundaries. A compact measurement model is

$$\tilde{s}_t = \mathrm{wrap}_{[-\lambda/2,\lambda/2)}(e_t) + \eta_t^{\mathrm{meas}} + \eta_t^{\mathrm{GKP}}. \tag{2}$$

The two noise terms summarize effective readout and finite-squeezing contributions in the software model. They are not fitted to a calibrated finite-energy GKP device channel in this study. The wrap operation is the essential ambiguity: the same reported syndrome can correspond to different lattice branches. If the posterior mass is concentrated near one branch, a local affine estimator is a reasonable low-latency approximation. For jointly Gaussian local variables e and s, the linear-MMSE estimate is

$$\hat{e} = \mu_e + \Sigma_{es}\Sigma_{ss}^{-1}(s - \mu_s) = Ks + b, \qquad K = \Sigma_{es}\Sigma_{ss}^{-1}, \quad b = \mu_e - K\mu_s. \tag{3}$$

This derivation motivates the affine fast path while also defining its limits. More explicitly, fix a lattice branch $m_t \in \mathbb{Z}^2$ and define the unwrapped local residual $r_t = e_t - \lambda m_t$. Conditional on the effective noise state θ_t^{noise} and on this branch assignment, suppose the local pair (r_t, s_t) is approximated by a joint Gaussian with moments $(\mu_r, \mu_s, \Sigma_{rs}, \Sigma_{ss})$. The minimum-mean-square affine correction is then

$$\hat{r}_t = \mathbb{E}[r_t \mid s_t, \theta_t^{\text{noise}}, m_t] \approx \mu_r + \Sigma_{rs}\Sigma_{ss}^{-1}(s_t - \mu_s) = K(\theta_t)s_t + b(\theta_t). \tag{4}$$

The affine rule used by the fast path is therefore not introduced as a universal wrapped decoder. It is the single-branch local conditional-mean surface that remains after the slow loop has compressed recent syndrome statistics into a small committed parameter bank. The scientific question in this paper is whether such a surface can remain useful under controlled drift while preserving a deterministic low-latency datapath.

Proposition 1 (Local affine fast path)**.** *Fix an effective noise state θ and a lattice branch m. Let (r, s) denote the corresponding unwrapped local residual and wrapped syndrome, with finite second moments and nonsingular Σ_{ss}. Among all affine corrections $a(s) = As + c$, the unique minimum-mean-square correction is*

$$A^\star = \Sigma_{rs}\Sigma_{ss}^{-1}, \qquad c^\star = \mu_r - A^\star\mu_s.$$

If (r, s) is jointly Gaussian within this conditioned branch, this affine rule is also the conditional mean $\mathbb{E}[r \mid s, \theta, m]$.

Proof. For an affine correction $a(s) = As + c$, the mean-square risk is $\mathbb{E}\|r - As - c\|_2^2$. Differentiating with respect to c gives $c = \mu_r - A\mu_s$. Substituting the centred variables $\bar{r} = r - \mu_r$ and $\bar{s} = s - \mu_s$, the remaining risk is $\mathbb{E}\|\bar{r} - A\bar{s}\|_2^2$. The normal equations are $A\Sigma_{ss} = \Sigma_{rs}$, which yields $A^\star = \Sigma_{rs}\Sigma_{ss}^{-1}$ when Σ_{ss} is nonsingular. For a jointly Gaussian pair, the conditional mean is linear with the same coefficient and intercept. □

The proposition is intentionally local. It justifies the committed (K, b) surface as the optimal affine correction after a branch and an effective noise state have been compressed by the slow loop. It does not claim global optimality for wrapped, multi-branch posteriors, nor does it replace a finite-energy logical-channel simulation. Its role is to make the runtime contract mathematically explicit: a slow estimator may update the moments or their learned proxy, while the fast path only evaluates the affine map fixed by the proposition.

This problem setting is deliberately weaker than a full logical-channel simulation. A complete approximate-GKP analysis would track finite-energy envelopes, auxiliary-state noise, channel-specific loss or amplification, and the induced logical Pauli channel. The present model instead treats these effects through an effective wrapped displacement process and asks whether a runtime-constrained correction surface can adapt to controlled nonstationarity. That restriction makes the FPGA-facing question concrete: the fast path receives a wrapped syndrome and a committed affine parameter bank, not a full posterior over lattice branches.

The separation is important for comparing with fidelity-oriented GKP work. Channel-level studies ask how a physical loss, amplification or finite-energy model propagates to logical probabilities or process infidelity. This manuscript asks a narrower systems question: if the physical correction layer is restricted to a committed affine rule, can recent syndrome statistics keep that rule aligned with a drifting effective residual distribution? The answer is reported with a half-lattice boundary proxy and diagnostic channel-language surrogates. It is not reported as logical-channel fidelity

because the source data do not reconstruct the finite-energy output state or a complete logical process map.

The validity regime can be stated in operational terms. Let B_t be the event that the wrapped syndrome is assigned to the wrong lattice branch after conditioning on the recent histogram, and let c be the slow-loop update window. The affine path is expected to be useful when $\Pr(B_t \mid H_{t-c+1:t})$ is small, the local covariance of the conditioned residual is well described by a single mode, and the drift timescale is long enough that (K_t, b_t) remains informative between commits. It is expected to fail when posterior mass splits across adjacent lattice branches, when finite-energy tails place substantial probability near half-lattice boundaries, or when drift changes faster than the histogram window and commit cadence can track. These three failure modes motivate the paper's bounded evidence ladder: residual-MSE checks for local affine fit, boundary-crossing `final_ler_mean` for branch-cell errors, holdout drift stress tests for slow-loop lag and a hardware-facing datapath contract for future device validation. The controlled oracle and wrapped-Gaussian analyses in the Results section test this local validity regime rather than replacing a full wrapped decoder benchmark.

3.2 Effective drift state

The slow loop estimates a compact effective noise state rather than a full circuit-level model:

$$\theta_t^{\text{noise}} = (\sigma_t, \mu_{q,t}, \mu_{p,t}, \vartheta_t). \tag{5}$$

Here σ_t summarizes displacement scale, $\mu_{q,t}$ and $\mu_{p,t}$ represent mean bias, and ϑ_t captures covariance orientation. The current benchmark uses four hand-designed drift families: static bias with rotation, linear ramp, step change in variance/orientation and periodic drift. These scenarios test distinct adaptation demands but do not exhaust possible nonstationary GKP noise.

3.3 Evaluation target

The primary metric is `final_ler_mean`, a logical-error proxy measured at the end of a fixed software-in-the-loop simulation run. Lower values are better. Because the main benchmark data do not include confidence intervals or a formal predeclared holdout set, the main result is interpreted as a predeclared-protocol ranking, not as a broad distributional generalization claim. Controlled holdout stress tests are reported separately in the Results section, and repeat-expanded inferential confidence intervals or predeclared bootstrap intervals for the main rows remain required before stronger statistical language is appropriate.

Metric definition. For one software-in-the-loop simulation run, `final_ler` is the final value of the cumulative logical-error proxy reported by the fast-loop emulator,

$$\texttt{final\_ler} = \frac{N_X + N_Z}{T},$$

where N_X and N_Z are the numbers of q- and p-quadrature residual boundary crossings counted by `LogicalErrorTracker`, and T is the number of fast-loop rounds in the run. The table field `final_ler_mean` is the mean of this value across the available repeats for each scenario and mode; the associated `final_ler_sd` field is a descriptive standard deviation. This definition is a software-in-the-loop simulation protocol metric. It is not a hardware logical-error measurement, a logical-channel estimate, a confidence interval or a significance test.

The proxy follows the standard GKP intuition that a residual displacement past the half-lattice boundary corresponds to a logical branch error in the associated quadrature. Counting q- and p-boundary crossings therefore gives a software-level indicator of how often the fast path leaves the residual on the wrong side of the correctable cell. It does not capture the full logical channel, correlations across repeated correction rounds, leakage, auxiliary state noise or outer-code behavior. Those omissions are why the metric is reported as `final_ler_mean` rather than as logical-channel fidelity or hardware logical-error rate.

The relation between the reported proxy and the channel-style surrogate can be made explicit. For round t, define

$$X_t = \mathbb{1}\{|r_{q,t}| > \lambda/2\}, \qquad Z_t = \mathbb{1}\{|r_{p,t}| > \lambda/2\},$$

where $r_{q,t}$ and $r_{p,t}$ are the residual displacements after the committed affine correction. The reported run-level proxy is

$$\texttt{final\_ler} = \frac{1}{T}\sum_{t=1}^{T}(X_t + Z_t).$$

The union event used in the surrogate channel table is

$$U_t = \mathbb{1}\{X_t \vee Z_t\} = X_t + Z_t - X_t Z_t.$$

Thus $\mathbb{E}[U_t]$ and $\mathbb{E}[X_t + Z_t]$ are linked but not identical: a simultaneous q/p crossing contributes once to U_t and twice to `final_ler_mean`. In expectation, $p_{\text{any}} \le \mathbb{E}[X_t + Z_t] \le 2p_{\text{any}}$. This is why the manuscript reports `final_ler_mean` as the primary software metric and uses the surrogate channel language only as an interpretive bridge from boundary events to Pauli-like categories. A genuine finite-energy logical-channel fidelity would require propagating the corrected approximate-GKP state or an equivalent logical process map, not only counting residual boundary events [**?**, **?**, **?**].

3.4 Boundary sensitivity and surrogate channel language

The half-lattice boundary also gives a compact theoretical bridge between residual-scale metrics and channel-style language. If a residual quadrature is approximated locally as $r \sim \mathcal{N}(0, \sigma_r^2)$, the probability of a single-quadrature boundary crossing is

$$p_{\text{cross}}(\sigma_r) = \operatorname{erfc}\left(\frac{\lambda/2}{\sqrt{2}\sigma_r}\right). \tag{6}$$

Assuming independent q and p residuals with the same local scale gives

$$p_{\text{any}} = 1 - (1 - p_{\text{cross}})^2. \tag{7}$$

This calculation is not a finite-energy logical-channel simulation. It is a diagnostic bridge: it explains why residual variance near the decision boundary can change a boundary-crossing metric sharply, and why `final_ler_mean` and residual MSE are related but not interchangeable.

3.5 Metric interpretation

The notation above is tied to a concrete software implementation, but only at the level needed for a scoped protocol claim. The codebase uses the same GKP lattice constant $\lambda = \sqrt{2\pi}$ for syndrome wrapping, half-lattice logical boundaries and fast-loop histogram limits. The measurement model includes finite-energy and readout terms through a compact configuration, and the fast-loop emulator records residual accumulation with the same half-lattice logical-error rule. These anchors support

the statement that `final_ler_mean` is a protocol-defined logical-error proxy for the current software-in-the-loop simulation setting. They do not turn the benchmark into a complete finite-energy GKP physical model, a wrapped-posterior decoder comparison, a holdout drift distribution or a hardware measurement. The current source-data checks are consistency checks between manuscript tables, code paths and generated figures; they are not physical validation.

4 Method

4.1 Overview

The method decomposes drift-adaptive correction into three modules (Fig. 1). The fast path applies the active affine correction. The teacher or calibration module estimates a low-dimensional noise state from recent histograms. The runtime controller stages, clips, smooths and commits candidate parameters into the bank consumed by the fast path.

Each module has a distinct scientific and engineering role. The fast path is the object that must eventually meet a hard timing budget, so it is deliberately limited to clipping, fixed-point arithmetic, a 2×2 matrix-vector operation and status logging. The slow estimator is the object that absorbs nonstationarity, so it is allowed to use windowed histograms, teacher features or statistical summaries unavailable to a single-shot combinational datapath. The runtime controller is the safety interface between them: it prevents an unstable estimate from becoming an active correction until the candidate surface has passed clipping, quantization and commit checks. This separation lets the paper compare estimator families without changing the timing-critical operation being validated.

4.2 Algorithmic protocol

The protocol-level object is a sequence of committed affine surfaces rather than a single neural prediction. At each slow-loop update, the implementation uses the recent syndrome window $H_{t-c+1:t}$, its time differences and any available teacher or statistical state to propose a candidate $(K_t^{\mathrm{cand}}, b_t^{\mathrm{cand}})$. The candidate is clipped, optionally smoothed, checked against fixed-point and saturation limits, and staged in an inactive parameter bank. A commit event makes the candidate the active surface only at a safe epoch boundary. Between two such commits, every shot uses the same active surface through Eq. (1).

This organization makes the estimator interchangeable but keeps the fast-loop contract fixed. A teacher-anchored residual branch, a statistical calibration rule or a future estimator may differ in how it proposes $(K_t^{\mathrm{cand}}, b_t^{\mathrm{cand}})$, but the logged runtime state is the same: active K, b, staged K, b, clipping and saturation status, commit acknowledgement, stale-parameter counters and the resulting residual boundary events. These fields are the minimum information needed to reproduce the software protocol and to compare a future board implementation against the same source trajectory. The present manuscript evaluates this protocol in software; it does not report board commit latency or source-vs-board agreement.

4.3 Affine fast path

Given the measured syndrome s_t, the fast path computes

$$s_t^{\mathrm{clip}} = \mathrm{clip}(s_t, -s_{\max}, s_{\max}), \tag{8}$$

$$\Delta_t^{\mathrm{raw}} = K_t s_t^{\mathrm{clip}} + b_t, \tag{9}$$

$$\Delta_t = Q(\mathrm{clip}(\Delta_t^{\mathrm{raw}}, -\Delta_{\max}, \Delta_{\max})), \tag{10}$$

where $Q(\cdot)$ denotes fixed-point quantization. A hardware implementation can represent K_t, b_t, clipped syndromes and residual corrections in a selected $Q_{m,n}$ format, expose saturation flags, and route the active parameter bank through a small matrix-vector datapath. The fast-path latency model is correspondingly simple:

$$L_{\text{fast}} = L_{\text{wrap}} + L_{\text{matvec}} + L_{\text{clip}} + L_{\text{bank}}. \tag{11}$$

This module is needed because the per-shot QEC path must remain small enough for deterministic execution and independent validation. Its advantage is not that affine decoding is globally optimal, but that all more complex estimation can be moved outside the fast timing boundary.

4.4 FPGA-facing datapath contract

The hardware-facing object in this study is a datapath contract, not a measured device implementation. The contract has five parts: a wrapped two-quadrature syndrome input; an active bank containing K_t, b_t and clip thresholds; a fixed-point affine matrix-vector operation; a corrected displacement output; and status flags for clipping, overflow, stale parameters and bank commits. This decomposition is useful because every fast-path output can be checked against a software reference with explicit input vectors and parameter-bank state. It also makes the negative comparison precise: a per-shot wrapped-posterior decoder must evaluate multiple branch hypotheses inside the timing boundary, whereas the proposed path evaluates one clipped affine rule and leaves posterior or neural estimation to the slow loop.

4.5 Teacher-anchored residual calibration

The teacher estimator maps a recent histogram window $H_{t-c+1:t}$ to an effective state $\hat{\theta}_t^{\text{teacher}}$, then to teacher parameters through the affine mapper:

$$(K_t^{\text{teacher}}, b_t^{\text{teacher}}) = \text{ParamMapper}(\hat{\theta}_t^{\text{teacher}}). \tag{12}$$

The learned branch predicts only a bounded residual bias term,

$$\widehat{\delta b}_t = f_\phi(H_{t-c+1:t}, \Delta H_{t-c+2:t}, z_t^{\text{teacher}}), \tag{13}$$

followed by clipping and smoothing:

$$b_t = \text{EMA}\Big(b_t^{\text{teacher}} + \text{clip}(s_b \widehat{\delta b}_t, -b_{\max}, b_{\max})\Big), \qquad K_t = K_t^{\text{teacher}}. \tag{14}$$

The motivation is to let the teacher define a stable physical calibration surface while the residual branch handles systematic mismatch left by the teacher. The design is intentionally conservative: the neural network never directly replaces the fast GKP correction law.

The residual branch used in the reported experiments is deliberately small and runtime-aligned (Table 9). Its inputs are the same window-level quantities available to the slow loop, and its labels are residual affine-bias corrections rather than direct logical decisions. This choice reduces leakage from the evaluation objective into the per-shot path: the model learns how to correct the teacher's committed bias surface, while the fast loop still receives only (K, b).

4.6 Stage-and-commit runtime contract

The slow loop does not mutate active parameters immediately. It stages a candidate (K, b) in an inactive bank, validates coefficient bounds, quantization error and saturation risk, and commits at a safe epoch boundary. The contract exposes observable quantities: update cadence, stale-parameter counters, commit acknowledgements, rollback events, overflow and saturation. In this study, these quantities are evaluated only in software-in-the-loop simulation and supporting software-runtime checks. Board-level commit latency, source-vs-board agreement and resource use remain unmeasured.

4.7 Supplementary statistical calibration

The same affine contract can host non-neural estimators. A scoped statistical-calibration branch computes a bias correction directly from recent syndrome statistics and emits parameters through the same clipping and commit path. This branch is useful scientifically because it tests whether the affine calibration contract, rather than the CNN architecture alone, explains the observed gains. It is reported as a supplementary analysis in this manuscript and remains separate from the main software-in-the-loop simulation comparison.

5 Experimental Design

5.1 Claim-to-test map

The experiments are organized around four claims, each with a different validation burden. The first claim is empirical: a slow-loop residual branch can lower the protocol-defined `final_ler_mean` of a deterministic affine GKP fast path under controlled drift. This is tested by the predeclared five-mode software-in-the-loop simulation benchmark. The second claim is mechanistic: the advantage is consistent with adaptive calibration of a low-dimensional affine surface rather than with a single irreplaceable neural architecture. This is tested by feature ablations, six-seed mechanism summaries and the supplementary statistical-calibration branch. The third claim is theoretical and diagnostic: the affine rule is a locally meaningful approximation when a branch and effective state are fixed, but it is not a global wrapped-posterior decoder. This is tested by oracle-affine and wrapped-Gaussian checks. The fourth claim is hardware-facing but not yet hardware-validated: the committed affine path is a small datapath that can be subjected to fixed-point, source-vs-board and latency measurements. This is supported only by operation counts, software fixed-point parity and a future measurement protocol.

This map prevents a common reviewer failure mode in algorithmic QEC papers: using one successful metric to imply several stronger ones. The main benchmark can support a controlled drift ranking, but not logical-channel fidelity, statistical significance, real-time FPGA latency or end-to-end surface-GKP overhead. Those stronger claims require the separate evidence classes listed in Tables 4 and 29.

5.2 Predeclared simulation benchmark

The main experiment uses a predeclared four-scenario, five-mode software-in-the-loop simulation protocol. The five main modes are an extended Kalman filter, an unscented Kalman filter, a constant-mean affine baseline, an RLS residual-b baseline and the teacher-anchored hybrid residual-b branch. All reported main rows use the same predeclared scenario matrix and paired-seed/repeat setting. This benchmark tests adaptive calibration laws under matched synthetic drift trajectories. It does not test a real FPGA board, a default embedded runtime, or unseen drift families.

5.3 Comparator ladder and stress-test roles

The comparison design is intentionally layered because no single baseline can answer all reviewer-facing questions for a drift-adaptive GKP fast path. The main software-in-the-loop simulation table compares deployable calibration laws under the same runtime interface. The controlled oracle and wrapped-Gaussian checks ask whether the affine form is locally meaningful when the effective state is known. The holdout stress tests ask whether update latency, discontinuities or faster-than-window drift can erase the benefit of a better affine surface. The statistical-calibration extension asks whether the contract is tied to a particular neural estimator. Table 10 summarizes these roles before the results are reported.

5.4 Ablation and mechanism materials

The ablation materials remove or alter histogram-delta inputs and teacher-side features. The six-seed mechanism pack summarizes descriptive cross-seed behavior and a lower-clip intervention. These materials are included to explain the main result conservatively. They are not designed to establish causal mechanism closure.

5.5 Hardware-validation protocol

Hardware-facing validation is specified as a future-measurement protocol rather than as an available result. A board-level validation study would report the board platform, bitstream or firmware hash, AXI/DMA interface, fixed-point format, fast-path latency distribution, slow-loop update time, commit acknowledgement latency, resource use, power and source-vs-board numerical agreement. Until those measurements exist, hardware is treated as a validation target rather than as a completed result.

6 Results

6.1 Hybrid residual calibration ranks lowest in the predeclared simulation set

Under the predeclared four-scenario protocol, the hybrid residual branch has the lowest mean `final_ler_mean` in all four tested scenarios, while UKF is the runner-up in each case (Table 11). Relative to UKF, the hybrid branch reduces mean `final_ler_mean` by 1.75% in static-bias drift, 2.89% in linear ramp drift, 2.80% in the step-change scenario and 1.85% in periodic drift, for an average reduction of 2.32% across the tested set. This is the first rung in the comparator ladder in Table 10: it compares deployable calibration modes under matched software conditions before the later diagnostic baselines test oracle access, posterior branches, holdout stress and estimator interchangeability. The paired raw rows are directionally consistent across the two available repeats in each scenario (Tables 12, 13 and 14). The result supports the method claim that slow-loop residual calibration can improve a deterministic affine GKP fast path under the current software-in-the-loop simulation protocol. The current run contains two repeats per scenario and mode. The corresponding standard deviations, paired deltas and paired-bootstrap envelope are therefore reported only as descriptive uncertainty markers; they should not be interpreted as confidence intervals, standard errors or hypothesis-test evidence.

To make the ranking auditable as a magnitude statement rather than only a winner label, we also report the absolute UKF-minus-Hybrid margin and its scale relative to the larger of the two reported descriptive standard deviations (Table 14). This ratio is a source-data readout over the current paired rows, not a significance statistic. It shows that the hybrid advantage is larger than

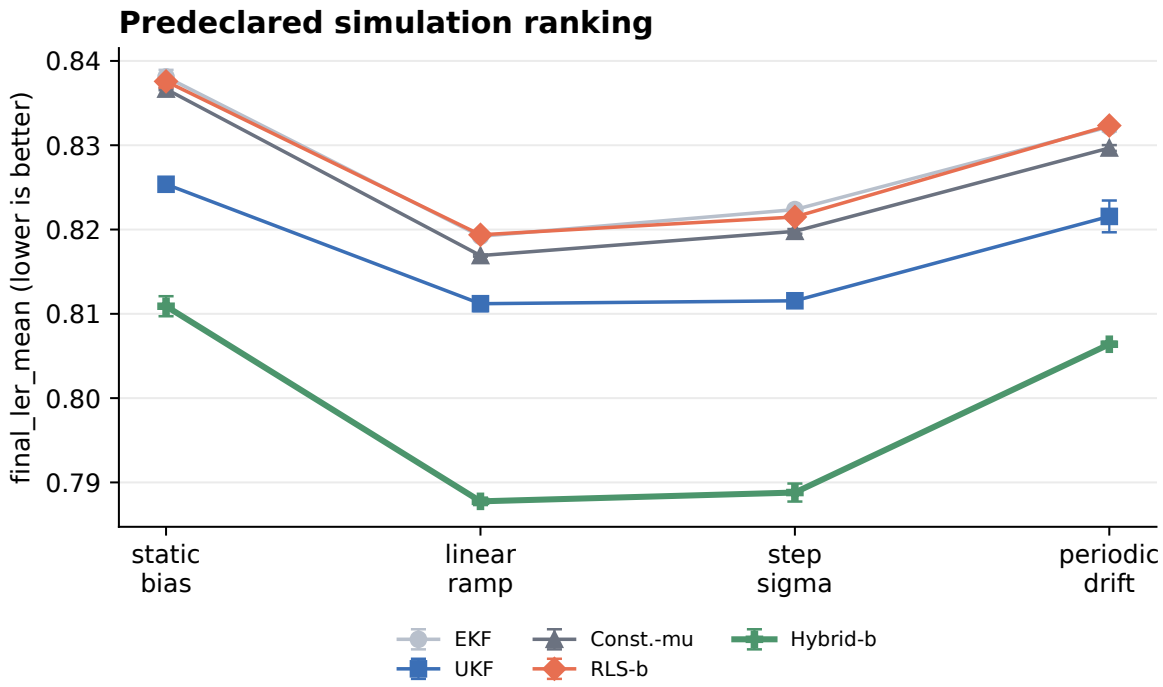

Figure 2: Main software-in-the-loop simulation comparison. The plot visualizes the existing `final_ler_mean` means from Table 11; lower is better. Error bars show descriptive standard deviations across the two available repeats per scenario and mode. The plot supports a predeclared four-scenario software-in-the-loop simulation ranking only. Paired repeat deltas are provided in Table 12, the descriptive resampling envelope is provided in Table 13, and source-data-backed advantage margins are provided in Table 14; all are bounded by the accompanying source data. Inferential confidence intervals, formal hypothesis tests and unseen drift families remain future evidence requirements.

the run-to-run descriptive SD reported for the two methods in each predeclared scenario, while preserving the same $n = 2$ statistical boundary as the main table.

A machine-readable benchmark-expansion protocol accompanies these descriptive paired rows. It specifies a Phase A repeat-expanded UKF-versus-Hybrid anchor comparison with at least 12 and a target of 16 paired repeats per scenario, while retaining the current five-mode table as the anchor result; a Phase B holdout lane for random-walk, burst/reset and faster-than-window drift families; and a Phase C software-provenance/runtime-reporting layer. This protocol is a planning artifact only. The present manuscript still does not report inferential intervals, formal hypothesis tests or an executed holdout benchmark.

As a runner-level feasibility check for this planned expansion, we executed a smoke-length UKF-versus-Hybrid pilot over all four predeclared scenarios with paired seeds and two repeats. The pilot completed all requested rows and preserved the same direction as the main benchmark, but it used the shortened smoke timing rather than the main benchmark timing. It is therefore reported only as a feasibility and missing-row check for the expansion route (Table 15), not as additional performance evidence.

6.2 Controlled oracle and wrapped-Gaussian checks separate estimator and decoder limits

These controlled baselines answer a narrow reviewer question: whether the affine form is obviously dominated by a known-state wrapped posterior rule in a local one-step setting. They are sanity checks for the fast-path approximation, not substitutes for a formal known-noise benchmark.

To isolate whether the affine fast path is limited by the affine form itself, by the upstream estimator or by the absence of a posterior branch model, we added a small controlled local-Gaussian analysis. The analysis samples 120,000 one-step errors for four representative drift states, compares a fixed nominal affine rule with an oracle affine rule that is given the state $(\sigma, \mu_q, \mu_p, \vartheta)$, and adds one-step wrapped-Gaussian posterior-mean and MAP branch baselines.

The result is deliberately narrower than the main benchmark. The oracle affine rule reduces residual MSE relative to the fixed nominal affine rule in all four controlled states, with the largest reduction in the post-step state (Table 16). The wrapped-Gaussian posterior mean is slightly better than oracle affine only in the static state; in the ramp, step and periodic states it has higher MSE, and the MAP branch rule is weaker still. This mixed outcome is important. It shows that known-state affine correction captures a real local modelling advantage, while a wrapped-posterior comparator cannot be treated as solved by simply adding a branch prior to the one-step setting. A sequence-level wrapped baseline, a holdout drift set and `final_ler_mean`-level scoring remain required before making stronger decoder-comparison claims.

6.3 A residual-boundary channel surrogate clarifies what `final_ler_mean` does and does not measure

The half-lattice crossing rule can be written in channel language without claiming a full logical-channel reconstruction. For each controlled local-Gaussian state, we decomposed residual boundary events into q-only, p-only, simultaneous q/p and no-crossing outcomes. This gives a Pauli-channel-style surrogate

$$p_I^{\mathrm{surr}} = 1 - p_{\mathrm{any}}, \quad p_q^{\mathrm{surr}}, \quad p_p^{\mathrm{surr}}, \quad p_{qp}^{\mathrm{surr}},$$

where p_{any} is the probability that the residual crosses at least one half-lattice boundary. If this decomposition is treated only as a qubit Pauli-channel surrogate, it induces $F_{\mathrm{avg}}^{\mathrm{surr}} = (1 + 2p_I^{\mathrm{surr}})/3$. The manuscript does not use this quantity as a finite-energy GKP logical-channel fidelity; it is included in the source data only to make the relationship between residual-boundary scoring and fidelity language explicit.

Table 17 reports the safer quantity, p_{any}. This union rate is related to, but not identical with, the manuscript `final_ler_mean` definition: `final_ler_mean` counts q- and p-boundary crossings separately, so a simultaneous q/p event contributes twice, whereas p_{any} asks whether at least one Pauli-like residual-boundary event occurred in the sample. The static and ramp states have no observed affine crossings in 120,000 controlled samples, whereas the post-step and periodic states expose low but nonzero residual-boundary events. The wrapped-Gaussian MAP branch rule has higher surrogate crossing probability in all four states, again showing that branch-aware posterior logic needs a tuned sequence-level protocol before it can serve as a stronger comparator. The table supports only a residual-boundary event decomposition; it does not estimate leakage, finite-energy envelopes, auxiliary-state noise, correlated rounds or process fidelity.

The same source data can also be read at the method level (Table 19). This aggregation is useful for review because it separates the question "which method has the larger residual-boundary channel surrogate?" from the per-state event decomposition. Across the four controlled local-Gaussian states, the fixed and oracle affine rows have the lowest mean p_{any}, while the wrapped posterior MAP

row has the largest worst-state surrogate crossing rate. This is still a lattice-level residual-boundary sanity summary, not a finite-energy channel measurement.

6.4 Sequence-level controlled baselines expose posterior-branch risk under drift

We next repeated the oracle and wrapped-Gaussian comparison over short drift sequences rather than isolated samples. Each scenario used 384 independent sequences of 512 steps, the same local-Gaussian state families as the main controlled analysis, and the same half-lattice residual-boundary proxy used in the manuscript metric. The sequence result preserves the one-step conclusion that known-state oracle affine lowers residual MSE relative to a fixed nominal affine rule, but it also exposes a practical risk of posterior-branch rules: the wrapped-Gaussian posterior mean and MAP references produced higher sequence-level crossing proxies in the ramp, step and periodic states (Table 20).

This result does not establish that affine decoding is globally superior to wrapped decoding. It shows that a branch-aware posterior comparator must be implemented and tuned as a sequence-level decoder, not inserted as a naive one-step branch rule. For this manuscript, the result strengthens the engineering rationale for a calibrated affine fast path while keeping the stronger-baseline requirement open.

6.5 Holdout drift stress tests expose adaptation space and lag sensitivity

We next tested three drift patterns that were not part of the main four-scenario comparison: a clipped random walk in the effective noise state, two burst/reset intervals, and an oscillatory drift faster than the slow-loop commit window. The analysis used the same local-Gaussian displacement model and the same 384-by-512 sequence shape as the controlled sequence study. It compared a fixed nominal affine rule, an oracle affine rule, a lagged-affine reference with a 64-step commit interval and a 32-step lag, and the same one-step wrapped-Gaussian posterior references.

The holdout stress test is informative because the half-lattice crossing proxy is rare in these controlled samples. The affine rows have the same observed crossing rate within each holdout scenario, so residual MSE is the more sensitive stress-test statistic. The same source data also report a residual-boundary Pauli-style average-fidelity surrogate; the oracle-affine values remain near unity because crossings are rare, but this is still not a finite-energy logical-channel fidelity. Oracle affine gives the lowest residual MSE in all three holdout families (Table 21), showing that a correctly updated affine surface still has adaptation headroom outside the main drift set. The lagged-affine row is deliberately mixed: it approaches oracle behavior under random-walk drift, but it is worse than the fixed rule under burst/reset and faster-than-window oscillation. This failure mode is a useful constraint on the method. It indicates that the affine fast path is not the limiting idea by itself; stale parameter commits can erase the adaptation benefit under rapid or discontinuous drift.

The result does not prove generalization of the trained residual branch, because the lagged row is a known-state slow-commit reference rather than the CNN calibration module. It also does not provide confidence intervals or a formal expanded benchmark. Its role is narrower: it adds an unseen-drift stress layer that makes the proposed fast-path contract more falsifiable and turns drift-tracking latency into a measurable design variable.

6.6 Commit-lag sweeps turn stale parameters into a design variable

The fixed 32-step lag in Table 21 is only one point on a slow-loop design surface. We therefore swept the commit lag over $0, 8, 16, 32, 64$ and 128 simulation steps while keeping the commit interval fixed

at 64 steps. The sweep reuses the three holdout drift families and shared sampled errors within each family, so it is a controlled stale-parameter diagnostic rather than a new benchmark.

The resulting pattern is deliberately not over-simplified (Table 22). Under random-walk drift, longer lag monotonically increases residual MSE while still remaining below the fixed nominal affine reference. Under burst/reset and faster-than-window drift, the lag surface is non-monotone because the stale state can occasionally land closer to a later reset or oscillation phase than a recently committed state. This is precisely why commit latency should be measured rather than inferred: the fast affine surface remains cheap, but the slow-loop commit policy is part of the method's empirical contract.

6.7 Analytical operation counts quantify the fast-path cost argument

The cost advantage of the affine fast path is an analytical datapath argument, not yet a measured FPGA-timing result. The count assumes a two-quadrature syndrome input, a 2×2 affine matrix, one bias vector, a single active parameter bank and one correction per fast-loop shot. It counts arithmetic and small-state storage on the latency-critical path, while excluding the slow-loop estimator, memory-controller implementation details and clock-frequency closure. Under this model, the affine rule uses one branch, four multiplications, four additions and no exponential or division operations (Table 23). A 3-by-3 wrapped-Gaussian MAP reference increases the multiplication count by 12.25-fold and the stored state by 5.5-fold, while a posterior-mean reference requires softmax-like nonlinear operations in addition to a larger arithmetic footprint. This comparison supports the design choice to keep posterior or learned estimation outside the latency-critical correction path. It does not claim timing closure, resource use, power or source-vs-board agreement. Together with the fixed-point and runtime-counter checks below, it provides the software evidence currently available for the datapath contract in Table 8.

6.8 Software fixed-point emulation bounds numerical degradation

The affine fast path is intended to be representable as fixed-point arithmetic, so the manuscript also checks numerical degradation before any board claim is made. We compared the floating-point affine runtime with the same runtime under Q4.20 fixed-point emulation on 80,000 controlled local-Gaussian samples per state. The largest correction difference was 1.6×10^{-6}, no scenario changed the residual-boundary crossing rate, and no fixed-point quantization saturation occurred (Table 24). This supports the fixed-point feasibility of the affine datapath under the tested parameter range. It does not establish board timing, source-vs-board agreement, resource use or power.

6.9 Runtime counters make the software contract observable

The same software-in-the-loop simulation comparison records runtime counters for commit activity, cycle violations, overflow and saturation. These counters do not measure board latency or hardware reliability, but they make the stage-and-commit contract auditable in the software protocol. Across the scenario-level comparison rows, each mode applied about 900 commits on average, slow-update violations were zero, correction saturation was zero, and the observed overflow rate was approximately 2.5×10^{-3}, dominated by histogram-input saturation (Table 25). This result supports the narrower claim that the runtime contract exposes measurable software counters; it does not establish board commit timing, rollback reliability or source-vs-board agreement.

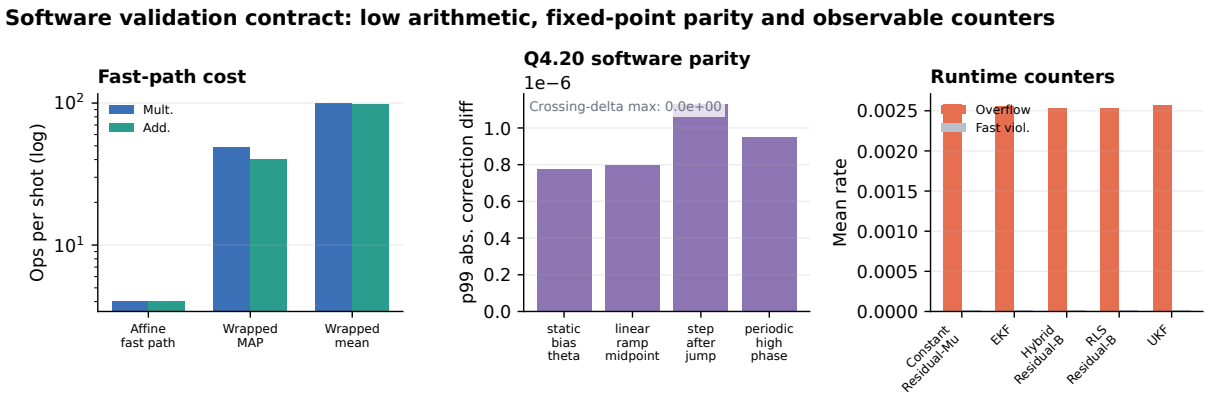

Figure 3: Software validation contract for the affine fast path. Left: analytical multiplication and addition counts compare the affine rule with one-step wrapped-Gaussian references. Middle: Q4.20 software fixed-point emulation shows small p99 correction differences and no observed residual-boundary crossing-rate change in the controlled sample set. Right: software-in-the-loop runtime counters make overflow and fast-cycle violations visible under the current protocol. The figure summarizes source-data-backed software checks; it is not FPGA synthesis, timing closure, power/resource measurement, source-vs-board agreement or board commit latency evidence.

6.10 Feature ablations constrain the mechanism story

The feature and teacher ablation table shows that input choices materially change performance in the tested scenario set (Table 26). Removing histogram deltas degrades performance relative to the full hybrid model, which supports the use of recent histogram dynamics. However, the no-teacher-parameters row has the best four-scenario average. The result therefore does not support a simple claim that teacher parameters are necessary or always beneficial. It supports a weaker but more credible interpretation: the residual calibration design is sensitive to feature composition, and the current mechanism remains descriptive.

6.11 Cross-seed intervention evidence remains descriptive

The six-seed mechanism pack indicates that an instability pattern appears across multiple seeds, but the tested lower-clip intervention is mixed and mostly harmful (Table 27). This result is valuable because it prevents an over-simple mechanism story. Residual amplitude and committed bias dynamics may participate in failures, but the current intervention does not show that lowering the residual clip is a general fix.

6.12 Supplementary statistical calibration supports the contract

The statistical-calibration rows are reported under a separate supplementary protocol rather than under the predeclared main ranking protocol (Table 28). They use the same affine runtime interface but answer a different question: whether a non-neural estimator can populate the committed (K, b) surface at all. They should therefore be read as contract evidence, not as evidence that the neural branch was beaten in a fair main-table comparison. A mature comparator claim would require a single predeclared protocol that fixes candidate generation, selection, repeats, uncertainty handling and reporting before seeing the outcomes.

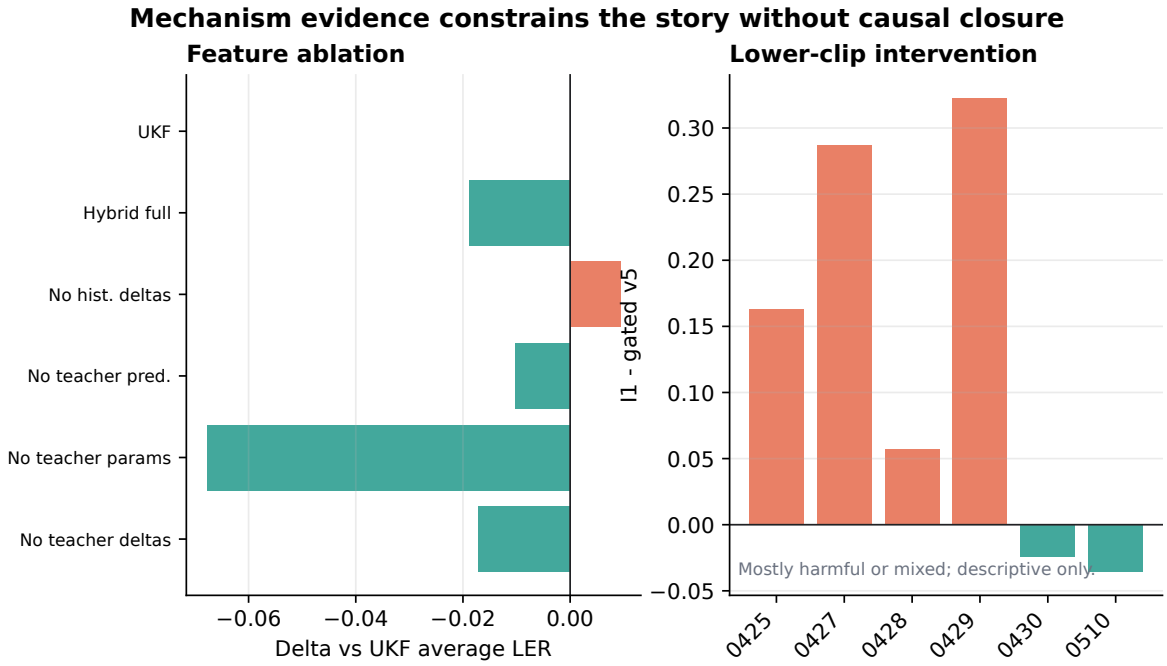

Figure 4: Ablation and descriptive mechanism evidence. Left: average LER change relative to UKF for the scoped feature ablation table. Right: lower-clip intervention effect across the six-seed mechanism pack. The figure supports feature sensitivity and a cautionary intervention reading; it does not establish causal closure, teacher necessity or a successful mitigation.

6.13 Required hardware and deployment measurements

The current manuscript does not contain real-board execution results. The available deployment-adjacent material supports only two narrower statements: selected preserved `.tflite` artifacts have executed in one isolated local software runtime, and no board path or device measurement is available. Table 29 defines the hardware measurements that a later experiment must fill.

7 Discussion

The present results support affine calibration as a systems structure for drift-adaptive GKP decoding. The strongest main-table evidence is not an unrestricted decoder comparison; it is the predeclared ranking of a hybrid residual branch under a controlled software-in-the-loop simulation protocol. Because scoped ablations show that teacher-parameter removal can be competitive and statistical calibration can populate the same interface, the result should be read as evidence for the affine calibration contract rather than for teacher-parameter necessity. This positions the contribution between two established lines of work: soft-information GKP decoding, which exploits analog syndrome structure, and real-time hardware decoders, which require bounded latency and verifiable datapaths. The distinctive design choice here is to let the slow loop absorb nonstationary syndrome statistics while the per-shot path remains a small affine matrix-vector update.

The supplementary statistical-calibration analysis sharpens this interpretation. If a simple non-neural estimator can be strong under the same runtime contract, the scientific object is the contract itself: recent syndrome statistics update a low-dimensional affine surface that the fast path can consume safely. The CNN branch is one useful slow-loop estimator, not the only possible

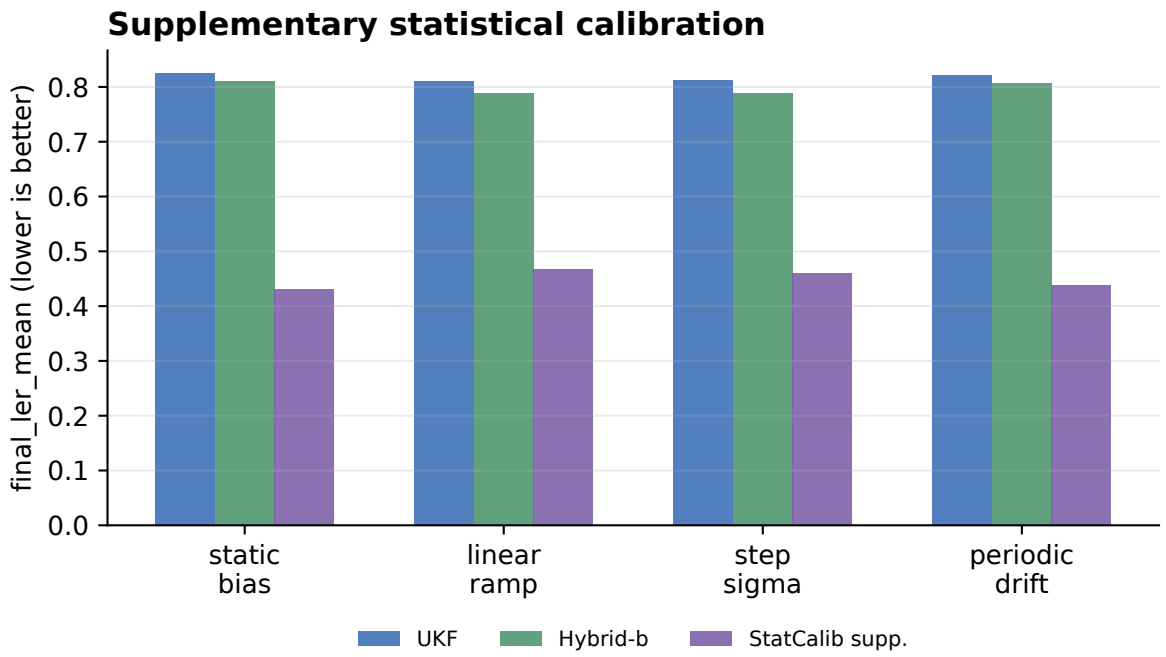

Figure 5: Supplementary statistical-calibration analysis. The plot compares UKF, hybrid-b and the statistical-calibration values already listed in Table 28. The result supports the broader affine calibration contract, but it must remain separate from the main comparison because it was not generated under the same predeclared ranking protocol.

estimator and not yet a final comparator.

The claimed advantage is therefore an interface-level advantage. Latency is addressed structurally by placing estimation in the slow loop and leaving the fast path as $\Delta_t = K_t s_t + b_t$; adaptation is supported by predeclared drift scenarios, paired descriptive deltas against UKF and controlled stress tests; cost and verifiability are supported by the analytical operation-count model, fixed-point software parity and the small source-vs-board target implied by a clipped affine datapath. These points motivate hardware validation, but they are not hardware results: board timing, resource use, power, commit reliability and source-vs-board vectors remain to be measured.

Several limitations are material for review. First, the drift evidence is still not a formal expanded benchmark. The random-walk, burst/reset and faster-than-window stress tests reduce the unseen-drift gap, but they use a controlled known-state stress protocol rather than the trained residual branch under a predeclared holdout benchmark. Second, the one-step and short-sequence oracle/wrapped-Gaussian checks separate part of the estimator error from the local affine model and show that a naive wrapped-posterior rule is not automatically stronger across drift states. They remain controlled local-Gaussian analyses, not formal benchmark baselines; known-noise affine, nearest-lattice and sequence-level wrapped-Gaussian benchmark protocols are still missing. Third, the current metric is not a logical-channel fidelity or process-infidelity estimate, so it cannot be compared directly with channel-level GKP analyses. The next validation step for that class of claim would be to report finite-energy logical-channel probabilities or channel infidelity under a specified physical channel, rather than reusing the residual-boundary surrogate. Fourth, the current main-result figure reports means with descriptive standard deviations and a descriptive paired-bootstrap envelope from only two repeats, not confidence intervals or paired significance tests. Fifth, the

hardware-facing path remains unvalidated: latency, resource, commit reliability and source-vs-board agreement are not yet measured. Finally, training reproducibility and default-environment runtime portability are not closed by the current evidence.

These limitations define the scope of inference. A stronger evidence package would upgrade the descriptive paired envelope into repeat-expanded or inferential uncertainty estimates, add one or more theoretical baselines, provide trained-branch holdout validation under a predeclared expanded protocol, repeat-expanded source data and row-level historical-run provenance closure, and either bit-accurate fixed-point measurements or real-board timing/resource results. If the FPGA path later succeeds, the physical advantage to test is concrete: an operation-count-light affine fast path, pending synthesis and resource measurement, with source-vs-board agreement, bounded update latency and measured timing headroom relative to a full per-shot learned decoder. Until then, the appropriate manuscript positioning is a simulation-first quantum-engineering systems method.

8 Conclusion

We presented a runtime-consistent affine calibration formulation for drift-adaptive GKP decoding. The method keeps the per-shot correction as the small affine rule $\Delta_t = K_t s_t + b_t$ while updating the affine parameters from recent syndrome statistics in a slower loop. Under a predeclared four-scenario software-in-the-loop simulation protocol, the main-table hybrid residual branch has the lowest mean `final_ler_mean` in all tested scenarios. Scoped ablations and mechanism materials support a conservative interpretation: histogram dynamics and feature design matter, and the more general object is the affine calibration interface rather than teacher-parameter necessity. Controlled oracle-affine and wrapped-Gaussian checks show that known effective-state affine parameters reduce residual MSE in local-Gaussian states, while a simple wrapped posterior does not dominate in either one-step or short-sequence settings. A supplementary statistical-calibration analysis further supports the affine-contract view, while remaining separate from the main comparison.

These results establish a simulation-first systems method, not a completed hardware demonstration. Hardware measurements, default runtime portability, expanded drift coverage, stronger baselines and statistical confidence intervals remain required before deployment-level or broad benchmark claims can be made.

9 Outlook

Future validation should proceed along three axes. First, statistical benchmark expansion should add repeat-expanded uncertainty estimates, trained-branch holdout drift families and formal known-noise affine, nearest-lattice or sequence-level wrapped-Gaussian baselines under a predeclared protocol. Second, physics and channel validation should test the lattice constant, syndrome wrap interval, logical boundary and `final_ler_mean` definition before using finite-energy logical-channel simulations to support channel-level fidelity claims. Third, hardware validation should replace requirements with board data only after a future experiment records device paths, bitstream or RTL hashes, DMA or MMIO evidence, latency distributions, resource use, power and source-vs-board numerical agreement. Together, these studies would test whether the affine calibration contract remains advantageous beyond the present software-in-the-loop simulation setting.

A Data, Code, and Reproducibility Availability

This appendix separates materials that support the present claims from materials that remain required for a stronger version of the study. Available software artifacts are sufficient for a software-in-the-loop simulation methods paper only if the manuscript preserves the validation scope in Table 31. They are not sufficient for a hardware-demonstration paper, a deployment paper or a broad decoder-comparison paper.

For a stronger empirical version of the study, the supporting package should include machine-readable result tables with seeds and uncertainty estimates, reproducible plotting scripts and figure manifests, exact configuration files for each drift family, runtime-artifact validation for the runner, a file-level source-data manifest for manuscript-facing tables and figures, a literature-metric crosswalk for related-work anchors, verified reference metadata and, when hardware language is used, board logs, bitstream hashes, fixed-point test vectors, latency measurements and resource reports. The present manuscript provides file-level hashes for its source CSV/JSON files and generator scripts, together with a row-level provenance manifest for the main software-in-the-loop simulation benchmark rows. This remains narrower than a recursive hash closure over historical run directories or a hardware provenance record. Until those stronger materials exist, hardware rows and broad reproducibility statements must remain future validation targets.

B Scope of Validation

C Benchmark Protocol and Source Data

A formal numerical comparison must make clear whether results come from a fixed protocol or from post-hoc selection of convenient runs. Table 33 therefore separates the protocol, the source artifact and the allowed manuscript use for each result class. The table is intentionally conservative: it supports a software-in-the-loop simulation manuscript, not a hardware or deployment claim.

D Theory-to-Configuration Bridge

The theory sections use deliberately compact notation: a syndrome stream s_t, an affine fast-path update $\Delta_t = K_t s_t + b_t$, and a slow calibration loop that updates (K_t, b_t) from recent syndrome statistics. The current evidence instantiates this notation through four controlled synthetic drift scenarios in the predeclared simulation benchmark. These scenarios are protocol probes of an effective calibration surface; they are not a complete physical noise model, a holdout drift distribution or a hardware environment.

The four scenario names should therefore be read as controlled synthetic probes. The static-bias scenario fixes an effective biased orientation state; the linear-ramp scenario tests monotone scale drift with orientation jitter; the step-change scenario tests an abrupt effective-state change; and the periodic-drift scenario tests oscillatory tracking. The protocol uses paired seeds, two repeats, four scenarios and five predeclared modes. This supports a scoped claim about runtime-consistent affine calibration under the predeclared simulation protocol; it does not support language about physical device drift, broad distributional generalization, confidence-level significance or real-board validation.

E Runtime Artifacts and Reproducibility Limits

The present study uses several artifact and runtime classes that must not be conflated. Table 35 records the manuscript-facing taxonomy. This classification does not replace a full reproducibility package, but it states which artifacts support the reported simulation results and which artifacts remain outside the reported hardware evidence.

Three validation gaps remain. First, the present source-data checks cover the main result table, paired-delta tables, ablation and mechanism tables, statistical-calibration rows, controlled oracle/wrapped-Gaussian checks, sequence-controlled baselines, holdout stress diagnostics, fast-path cost, fixed-point parity, runtime-discipline counters, residual-boundary surrogate rows and the metric-readiness matrix. They still do not cover every narrative scope, availability or requirements table in the manuscript. Second, the source-data manifest records file-level SHA-256 hashes for manuscript-facing source CSV/JSON files and generator or audit scripts, and a row-level provenance manifest traces the main software-in-the-loop simulation benchmark rows to scenario, mode, repeat, seed, source run path, config, runner and selected repeat-summary hashes. These manifests are still not a recursive hash closure over historical run directories and do not hash every large event log. Third, the hardware surface remains a readiness check until a future board experiment produces device logs, bitstream or RTL hashes, DMA evidence, latency/resource measurements and source-vs-board vectors.

F Statistical Treatment and Reproducibility

The current main result is a predeclared-protocol ranking with descriptive uncertainty, not a distribution-level statistical claim. The main benchmark uses paired seeds and two repeats per scenario and mode. The manuscript therefore reports means, descriptive standard deviations, paired deltas and a descriptive paired-bootstrap envelope only. It does not report inferential confidence intervals, p-values, standard errors or hypothesis-test evidence.

These limits also determine the claim language in the abstract and conclusion. The supported statement is that the hybrid residual branch has the lowest descriptive mean `final_ler_mean` in the predeclared software-in-the-loop simulation set, and that the evidence motivates a runtime-consistent affine calibration strategy. The present evidence does not establish statistical significance, robustness across unseen drift families, full reproducibility, cross-host portability, hardware validation or deployment readiness.

G Claim Scope and Source Data

The manuscript is organized around a small number of scoped claims. Each claim must remain tied to a specific result or source-data layer. Tables 38 and 39 record that relationship for manuscript assembly; they are not new experiments and do not strengthen any result by themselves.

H Limitations and Required Validation

The validation limits are stated explicitly before they are used to qualify the claims. Table 40 summarizes the principal limitations of the present study and the evidence required before each claim could be strengthened.

I Terminology

J Figure Source Data and Status

The figure set is tied to source-data tables and validation scope. The present manuscript uses Python/Matplotlib figures, and hardware validation is represented as a measurement-requirements table rather than as device data. The table below defines the intended claim, source data and status for each figure in the present manuscript numbering.

Comparison axis	Representative prior-work metrics	Relevance to this paper
Analog GKP information	Analog syndrome decoding has been reported to lower required squeezing from about 16.0 dB to 9.8 dB in surface-GKP settings [**?**, **?**].	The present work does not claim novelty in analog information. It uses recent analog histograms to update a low-dimensional affine correction surface.
Calibration-aware QEC	Prior calibration and drift-aware graph updates report gains ranging from percentage-level prior improvements to multi-fold mismatch reductions [**?**, **?**, **?**].	The calibrated object here is the physical-layer (K, b) surface consumed by the fast GKP correction path, not only an outer-code graph weight or decoder prior.
Logical-error and overhead targets	Surface-GKP studies report logical-failure and overhead targets such as 0.87% to 0.36% CNOT-failure reduction, 9.9 dB threshold behavior, and 291 modes plus 97 qubits versus 1457 bare qubits for a matched low-failure target [**?**].	The present metric is narrower: it is a protocol-defined q/p boundary-crossing proxy, not the same endpoint as prior-work LER or logical-failure rates. The result should therefore be read as a front-end calibration result, not as an end-to-end surface-GKP overhead claim.
Logical-channel fidelity and infidelity	Approximate-GKP analyses report logical-channel probabilities, loss-channel infidelity and near-optimal loss/amplification performance as channel-level figures of merit [**?**, **?**, **?**].	The current manuscript reports a residual-boundary Pauli-channel-style surrogate derived from q/p half-lattice crossings, but not finite-energy logical-channel fidelity or process infidelity. Fidelity-level claims would require channel-specific simulation, logical-channel probability estimates or process-infidelity reconstruction under a specified physical channel.

Table 2: External GKP and calibration comparison axes. The cited values are representative reported metrics from adjacent work, not normalized baselines for this study. They position the contribution and the missing validation without treating incompatible metrics as results of this manuscript. The companion literature metric crosswalk records the local literature-card figure, table, equation or card-level anchor used for each representative metric.

Comparison axis	Representative prior-work metrics	Relevance to this paper
Real-device learned decoding	Learned decoders have reported real-processor LER reductions using large experimental and synthetic data sets [**?**, **?**].	The present manuscript does not report real-processor learned-decoder LER. It uses software drift scenarios to test whether a learned slow loop can populate a deterministic affine surface.
Structured neural and soft decoders	Bosonic/GKP-like neural decoders, neural matching-weight predictors and hardware soft-information decoders provide related routes to improved LER or thresholds [**?**, **?**, **?**].	The present work does not claim novelty in neural decoding or soft information. Its distinction is that the neural component is confined to slow calibration of the affine physical correction surface.
Runtime pre-decoders and calibration-conditioned latency	AI pre-decoders and calibration-conditioned decoders report runtime anchors such as 38.78 μs sub-step decoding or 85–95 μs folded FiLM latency [**?**, **?**].	The current latency argument is only analytical and software-emulated. A future device study must measure the affine fast path rather than borrow these per-shot neural latency values.
Real-time FPGA decoders	Hardware decoders report deployment metrics such as 28–42 ns lookup latency and compressed lookup memory [**?**], 11.5 ns per measurement round [**?**], sub-μs round or feedback timing [**?**, **?**, **?**], 550 ns closed-loop latency and QPU-feedback integration [**?**], and scale, memory, area and power figures [**?**].	These works define the standard for hardware claims. This manuscript only specifies the required hardware measurement surface until the same classes of timing, resource and source-vs-board data are available.

Table 3: External learned-decoder and hardware-runtime comparison axes. The cited values are representative reported metrics from adjacent work, not normalized baselines for this study. They position the slow-loop calibration role and the missing hardware validation, not as results of this manuscript. The companion literature metric crosswalk records the local literature-card figure, table or card-level anchor used for each representative metric.

Metric axis	Current manuscript metric	Supported statement	Missing for stronger claim
Logical-error proxy	`final_ler_mean` means, paired descriptive deltas and a non-inferential paired-bootstrap envelope	Hybrid residual calibration has the lowest mean protocol-defined `final_ler_mean` in the tested drift scenarios.	More repeats, inferential confidence intervals, holdout drift and logical-channel reconstruction.
Logical-channel fidelity	Residual-boundary Pauli-channel-style surrogate plus method-level lattice sanity summary	The surrogate links q/p half-lattice crossing rates to channel language and reports mean/worst-state p_{any} and $F_{\mathrm{avg}}^{\mathrm{surr}}$ readouts without estimating finite-energy logical-channel fidelity or process infidelity.	Channel-specific finite-energy simulation or logical-channel tomography.
Drift robustness	Four scenario rankings plus controlled local-Gaussian checks, holdout drift stress tests and a commit-lag sweep	Controlled stress diagnostics expose where stale commits help or fail under random-walk, burst/reset and faster-than-window settings; they do not establish trained-branch holdout robustness.	A formal predeclared expanded benchmark, repeat-expanded uncertainty estimates, trained-branch holdout validation and board-measured stale-parameter latency distributions.
Fast-path cost and latency	Analytical per-shot operation count plus Q4.20 software parity	The affine correction has lower analytical arithmetic and state footprint than the wrapped-posterior references evaluated here, and fixed-point emulation introduces no residual-boundary crossing change in the controlled sample set.	FPGA synthesis, timing closure, resource and power measurements, source-vs-board agreement and board-level latency distribution.
Hardware-facing validation	Measurement requirements only	The manuscript defines what must be measured before making hardware claims.	Board logs, bitstream or RTL hash, DMA evidence, commit reliability and source-vs-board vectors.

Table 4: Metric-readiness matrix for the manuscript. The table separates reported metrics from external standards and future validation requirements; it does not convert the current `final_ler_mean` proxy into channel fidelity, statistical significance or hardware latency.

σ_r	Eq. squeezing (dB)	p_{cross}	p_{any}	Surrogate infidelity
0.15	16.48	0.000000	0.000000	0.000000
0.20	13.98	0.000000	0.000000	0.000000
0.25	12.04	0.000001	0.000001	0.000001
0.30	10.46	0.000029	0.000059	0.000039
0.35	9.12	0.000342	0.000685	0.000456
0.40	7.96	0.001729	0.003454	0.002303

Table 5: Analytical residual-boundary sensitivity for a zero-mean Gaussian residual model. The equivalent squeezing column is $-10\log_{10}\sigma_r^2$. The surrogate infidelity is $1-(1+2(1-p_{\mathrm{any}}))/3$, used only to connect boundary events to channel-style terminology; it is not a finite-energy GKP logical-channel fidelity or process-infidelity estimate.

Object	Current support	Manuscript use	Missing before stronger claim
Syndrome wrapping	Shared $\sqrt{2\pi}$ lattice constant and wrap interval $[-\lambda/2, \lambda/2)$.	Approximate GKP syndrome model.	Independent symbol-to-code validation.
Finite-energy and measurement noise	Configurable Δ, measurement efficiency, shot noise and ancilla-error terms in the software measurement model.	Approximate measurement effects only.	Full physical-noise calibration.
Effective drift	Synthetic $(\sigma, \mu_q, \mu_p, \vartheta)$ trajectories drive the software-in-the-loop simulation protocol.	Controlled drift probes.	Unseen or device-derived drift families.
`final_ler_mean`	Residual accumulation is scored against half-lattice logical boundaries in the fast-loop emulator.	Predeclared-protocol logical-error proxy.	Logical-channel validation and stronger baselines.
Hardware path	Board-facing rows specify future measurement requirements.	Future validation target.	Measured device logs and resource/timing data.

Table 6: Physics-model and metric scope. The current implementation anchors the notation used in the paper, but the evidence supports only a software-in-the-loop simulation protocol claim.

Step	Operation	Reproducible state recorded by the protocol
1	Accumulate recent syndrome statistics.	Windowed histogram $H_{t-c+1:t}$, histogram differences, active scenario index and fast-loop shot count.
2	Propose a candidate affine surface.	Candidate K_t^{cand}, b_t^{cand}, estimator family and any teacher or statistical state used to compute the proposal.
3	Apply runtime guards before exposure to the fast path.	Clipped and optionally smoothed K, b, saturation flags, overflow flags and the inactive bank selected for staging.
4	Commit only at a safe update boundary.	Commit acknowledgement, active-bank selector, stale-parameter counter and rollback status.
5	Correct every shot with the committed affine rule.	Clipped syndrome, quantized correction, residual displacement and q/p half-lattice boundary indicators.

Table 7: Reproducible algorithmic contract for affine runtime calibration. Different slow-loop estimators may populate the candidate surface, but all paths must expose the same committed K, b state and the same fast-loop log fields. These fields are sufficient for the software comparisons reported here and define the future source-vs-board trace needed for hardware validation.

Datapath element	Manuscript contract	Current support
Input state	Wrapped syndrome s_t, clip limits and active-bank selector.	Defined by the software fast-loop interface and metric audit.
Parameter bank	Active and inactive K, b banks with staged commit and rollback events.	Exercised by software-in-the-loop simulation runtime counters only.
Arithmetic	One 2×2 matrix-vector operation, one bias vector and bounded clipping in fixed-point form.	Analytical operation count and Q4.20 software fixed-point emulation.
Output check	Corrected displacement and residual-boundary crossing indicator.	Compared in software against the floating-point reference.
Device evidence	Timing, resource, power and source-vs-board agreement.	Not measured in the present study; required for a hardware claim.

Table 8: FPGA-facing datapath contract for the affine fast path. The table defines what a future board implementation would need to reproduce. The current manuscript supports the interface with software and analytical checks only; it does not report synthesis, timing closure or board measurements.

Item	Reported setting	Scope
Input tensor	Five 32×32 histogram windows, four histogram-delta planes, teacher state, teacher runtime bias and teacher temporal deltas, encoded as 21 spatial channels.	Slow-loop residual branch only.
Targets	Two residual-b labels, b_q and b_p, defined as target runtime bias minus teacher runtime bias.	Residual calibration, not direct logical decoding.
Dataset split	2048 runtime windows from the four drift scenarios, split 1638/204/206 into train/validation/test windows.	Split-level check only; not unseen-drift validation.
Model	One 3×3 convolution with 16 channels, ReLU, 2×2 average pooling, one 96-unit hidden layer and two outputs.	Reported float model architecture.
Training	Weighted MSE on normalized residual-b outputs, label weights $w_{b_q} = w_{b_p} = 2$, learning rate 2×10^{-3}, weight decay 10^{-4}, batch size 64, 36-epoch cap and patience 8.	Training description only; not full reproducibility proof.

Table 9: Method details for the teacher-anchored residual branch. The table documents the reported slow-loop CNN component used by the manuscript; it does not make the branch a per-shot neural decoder or establish holdout-drift generalization.

Comparator or diagnostic	Question addressed	Interpretation limit
EKF and UKF modes	Whether standard nonlinear filtering baselines are competitive under the same scenario matrix and metric.	They are not optimized full posterior GKP decoders or hardware implementations.
Constant-μ and RLS residual-b modes	Whether simple affine or online residual updates explain the result without the teacher-anchored branch.	They do not cover all possible adaptive-prior or known-noise estimators.
Teacher-anchored hybrid residual-b	Whether a slow-loop residual branch can improve the committed affine fast path in the predeclared protocol.	Two repeats per scenario support only descriptive ranking, not inferential significance.
Oracle affine checks	Whether the affine mapper has headroom when the effective drift state is known.	Oracle state access is a diagnostic upper reference, not a deployable estimator.
Wrapped-Gaussian mean and MAP checks	Whether a simple branch-aware posterior rule obviously dominates the affine correction in local controlled states.	One-step or short-sequence controlled references do not replace a formal sequence-level wrapped-decoder benchmark.
Holdout drift stress families	Whether random-walk, burst/reset and faster-than-window drift expose adaptation or stale-commit failures.	The stress rows use controlled known-state references, not trained-branch holdout generalization.
Supplementary statistical calibration	Whether the same affine interface can be populated by a non-neural estimator.	It supports estimator interchangeability, not promotion of statistical calibration as a mature main comparator.

Table 10: Comparator ladder for the simulation and diagnostic evidence. The table states what each comparator can and cannot establish, so that the reported `final_ler_mean`, residual-boundary and cost results are read at the proper validation level.

Scenario	EKF	UKF	Const.-μ	RLS-b	Hybrid-b
`static_bias_theta`	0.838110	0.825370	0.836658	0.837577	**0.810902**
`linear_ramp`	0.819200	0.811201	0.816911	0.819373	**0.787755**
`step_sigma_theta`	0.822365	0.811548	0.819784	0.821493	**0.788800**
`periodic_drift`	0.832192	0.821558	0.829670	0.832334	**0.806392**

Table 11: Predeclared four-scenario software-in-the-loop simulation benchmark. Entries are `final_ler_mean`; lower is better. The table supports a predeclared-protocol software-in-the-loop simulation ranking. Confidence intervals, formal hypothesis tests and a predeclared expanded holdout benchmark remain future evidence requirements.

Scenario	Mean Δ`final_ler_mean` vs UKF	Relative reduction	Minimum paired Δ
`static_bias_theta`	0.014469	1.75%	0.013953
`linear_ramp`	0.023446	2.89%	0.023017
`step_sigma_theta`	0.022748	2.80%	0.022440
`periodic_drift`	0.015166	1.85%	0.013571

Table 12: Paired descriptive UKF-vs-hybrid deltas from the two available repeats per scenario. Δ`final_ler_mean` is UKF minus Hybrid-*b*, so positive values indicate lower `final_ler_mean` for the hybrid branch. The table documents repeat-level directional consistency in the existing source data; it is not a confidence interval, *p*-value or distribution-level robustness test.

Scenario	n pairs	Mean Δ	Envelope low	Envelope high	Positive pairs
`static_bias_theta`	2	0.014469	0.013953	0.014984	2/2
`linear_ramp`	2	0.023446	0.023017	0.023875	2/2
`step_sigma_theta`	2	0.022748	0.022440	0.023056	2/2
`periodic_drift`	2	0.015166	0.013571	0.016762	2/2
`all_scenarios`	8	0.018957	0.016044	0.021892	8/8

Table 13: Descriptive paired-delta resampling envelope for the UKF-versus-hybrid comparison. Δ is UKF `final_ler_mean` minus Hybrid-*b* `final_ler_mean` for paired repeats, so positive values indicate lower hybrid `final_ler_mean`. The envelope is a non-inferential paired-bootstrap percentile summary over the available repeat-level deltas. With $n = 2$ per scenario it is reported only for source-data transparency and must not be read as a confidence interval, *p*-value, standard error or robustness claim.

Scenario	UKF mean	Hybrid mean	Mean Δ	Relative reduction	Positive pairs	Δ/ max SD
`static_bias_theta`	0.825370	0.810902	0.014469	1.75%	2/2	12.17
`linear_ramp`	0.811201	0.787755	0.023446	2.89%	2/2	27.00
`step_sigma_theta`	0.811548	0.788800	0.022748	2.80%	2/2	21.28
`periodic_drift`	0.821558	0.806392	0.015166	1.85%	2/2	8.05

Table 14: Source-data-backed descriptive UKF-minus-Hybrid advantage margins. Δ is UKF `final_ler_mean` minus Hybrid-*b* `final_ler_mean`; positive values therefore indicate lower hybrid `final_ler_mean`. The last column divides the mean paired margin by the larger reported descriptive SD of the UKF and Hybrid-*b* rows in the same scenario. This is an effect-size-style scale readout for auditability only and is not a confidence interval, standard error, *p*-value, significance test, expanded benchmark or hardware measurement.

Scenario	UKF `final_ler_mean`	Hybrid `final_ler_mean`	Δ	Relative reduction	Positive pairs
`static_bias_theta`	0.817940	0.801012	0.016927	2.07%	2/2
`linear_ramp`	0.840509	0.813909	0.026600	3.16%	2/2
`step_sigma_theta`	0.828792	0.816023	0.012769	1.54%	2/2
`periodic_drift`	0.810660	0.781844	0.028817	3.55%	2/2

Table 15: All-scenario runner smoke matrix for the planned repeat-expanded anchor comparison. Δ is UKF `final_ler_mean` minus Hybrid-*b* `final_ler_mean`, so positive values indicate lower hybrid `final_ler_mean`. The source run used the smoke-length configuration (120 slow-loop updates and 4.8×10^5 fast cycles), two paired repeats and the formal runner over all four predeclared scenarios. This table checks execution coverage and field availability for the expansion route only; it is not the main benchmark, not an expanded benchmark, not an inferential interval and not hardware evidence.

Controlled state	Fixed affine	Oracle affine	Wrapped mean	Wrapped MAP
`static_bias_theta`	0.049199	0.048095	**0.047955**	0.048203
`linear_ramp_midpoint`	0.061417	**0.060251**	0.062341	0.063105
`step_after_jump`	0.108216	**0.092541**	0.105815	0.110339
`periodic_high_phase`	0.081757	**0.076028**	0.082349	0.084002

Table 16: Controlled local-Gaussian oracle-affine and wrapped-Gaussian checks. Entries are one-step residual MSE; lower is better. The table uses the manuscript affine mapper for the affine rows and known-state wrapped-Gaussian posterior rules for the posterior rows. It is a sanity check for estimator and decoder limits, not a formal software-in-the-loop simulation benchmark, sequence-level wrapped decoder comparison, holdout-drift test or hardware measurement.

Controlled state	Fixed affine	Oracle affine	Wrapped mean	Wrapped MAP
`static_bias_theta`	0.000000	0.000000	0.000000	0.000017
`linear_ramp_midpoint`	0.000000	0.000000	0.000042	0.000183
`step_after_jump`	0.000467	0.000467	0.002417	0.006450
`periodic_high_phase`	0.000042	0.000042	0.000617	0.001867

Table 17: Residual-boundary Pauli-channel-style surrogate for the controlled local-Gaussian states. Entries are p_{any}, the probability of at least one q/p half-lattice residual crossing; lower is better. The source CSV also records q-only, p-only, simultaneous crossing and $F_{\mathrm{avg}}^{\mathrm{surr}}$ fields, but the table reports only the crossing probability to avoid presenting the surrogate as finite-energy logical-channel tomography or process fidelity.

Controlled state	Fixed affine	Oracle affine	Wrapped mean	Wrapped MAP
`static_bias_theta`	1.000000	1.000000	1.000000	0.999989
`linear_ramp_midpoint`	1.000000	1.000000	0.999972	0.999878
`step_after_jump`	0.999689	0.999689	0.998389	0.995700
`periodic_high_phase`	0.999972	0.999972	0.999589	0.998756

Table 18: Surrogate average-fidelity readout derived from the same residual-boundary event decomposition as Table 17. Entries are $F_{\mathrm{avg}}^{\mathrm{surr}} = (1 + 2p_I^{\mathrm{surr}})/3$, with $p_I^{\mathrm{surr}} = 1 - p_{\mathrm{any}}$. The table is included only to make the manuscript's fidelity language auditable against source data; it is not a finite-energy GKP logical-channel fidelity, process fidelity, hardware fidelity or outer-code logical-error estimate.

Method	Mean p_{any}	Worst state	Worst p_{any}	Mean $F_{\text{avg}}^{\text{surr}}$	Worst $F_{\text{avg}}^{\text{surr}}$
Fixed affine	0.000127	`step_after_jump`	0.000467	0.999915	0.999689
Oracle affine	0.000127	`step_after_jump`	0.000467	0.999915	0.999689
Wrapped mean	0.000769	`step_after_jump`	0.002417	0.999488	0.998389
Wrapped MAP	0.002129	`step_after_jump`	0.006450	0.998581	0.995700

Table 19: Method-level lattice sanity summary for the residual-boundary channel surrogate. Values aggregate the same controlled local-Gaussian states and source rows as Tables 17 and 18. The table is included to make the channel-language readout easier to audit; it is not finite-energy GKP logical-channel fidelity, process tomography, hardware fidelity or an outer-code logical-error estimate.

Scenario	Fixed affine	Oracle affine	Wrapped mean	Wrapped MAP
`static_bias_theta`	0.000000	0.000000	0.000000	0.000000
`linear_ramp`	0.000015	0.000015	0.000158	0.000575
`step_sigma_theta`	0.000310	0.000310	0.001221	0.003220
`periodic_drift`	0.000005	0.000005	0.000127	0.000519

Table 20: Sequence-level controlled baseline analysis. Entries are mean half-lattice residual-boundary crossing proxies across 384 sequences of 512 steps; lower is better. This is a controlled local-Gaussian sequence analysis, not the formal software-in-the-loop simulation benchmark, a confidence-interval run, a holdout drift test or hardware validation.

Holdout stress family	Fixed affine	Lagged affine	Oracle affine	Wrapped mean	Wrapped MAP	Oracle $F_{\text{avg}}^{\text{surr}}$
`random_walk_drift`	0.078869	0.073867	**0.072685**	0.078740	0.080857	0.999915
`burst_reset_drift`	0.066865	0.068622	**0.062144**	0.065936	0.067789	0.999780
`faster_than_window_oscillation`	0.068354	0.068890	**0.063745**	0.067485	0.068764	0.999969

Table 21: Controlled known-state holdout drift stress diagnostic. Entries are residual MSE across 384 sequences of 512 steps unless otherwise stated; lower residual MSE is better. $F_{\text{avg}}^{\text{surr}}$ is the Pauli-channel-style surrogate induced by the oracle-affine residual-boundary identity rate, not a finite-energy logical-channel fidelity. The lagged-affine row uses known-state affine parameters committed every 64 steps with a 32-step lag, and is a slow-loop lag reference rather than the trained residual branch. The table is a non-hardware stress test for the affine contract, not a formal expanded benchmark, confidence-interval analysis or process-fidelity estimate.

Holdout stress family	Lag 0	Lag 8	Lag 16	Lag 32	Lag 64	Lag 128
`random_walk_drift`	0.073831	0.073976	0.074130	0.074410	0.075173	0.076093
`burst_reset_drift`	0.067454	0.068193	0.068657	0.068694	0.068583	0.066913
`faster_than_window_oscillation`	0.069251	0.069406	0.068989	0.069039	0.068842	0.068249

Table 22: Commit-lag sweep for the lagged affine reference. Entries are residual MSE across 384 sequences of 512 steps; lower is better. The commit interval is fixed at 64 simulation steps and the columns sweep additional simulation-step lag. This table is a non-hardware stale-parameter diagnostic, not a measured FPGA-latency result, source-vs-board agreement, trained-branch holdout generalization, a confidence-interval analysis or logical-channel fidelity.

Decoder path	Branches	Mult.	Add.	Nonlinear ops	State scalars
Affine fast path	1	4	4	0	6
Wrapped MAP, 3x3 branches	9	49	40	0	33
Wrapped posterior mean, 3x3 branches	9	99	98	18	33

Table 23: Analytical per-shot operation-count model. The affine row counts the two-quadrature matrix-vector correction used in the manuscript fast path. The wrapped rows count one-step known-state 3-by-3 branch references. The table supports a low-complexity design rationale only; it is not FPGA synthesis, timing closure, power/resource measurement or hardware validation.

Scenario	Max diff	p99 diff	MSE delta	Crossing delta	Quant. sat.
`static_bias_theta`	0.000001	0.000001	0.000000	0.000000	0.000000
`linear_ramp_midpoint`	0.000001	0.000001	0.000000	0.000000	0.000000
`step_after_jump`	0.000002	0.000001	0.000000	0.000000	0.000000
`periodic_high_phase`	0.000001	0.000001	0.000000	0.000000	0.000000

Table 24: Software fixed-point parity check for the affine fast path. Entries compare Q4.20 fixed-point emulation against the floating-point runtime under a controlled local-Gaussian sample set. Max and p99 diff are absolute correction differences. MSE delta and crossing delta are fixed-point minus floating-point. This table is a numerical-emulation check only, not FPGA synthesis, timing closure, source-vs-board agreement or hardware validation.

Mode	Commits	Slow viol.	Fast viol.	Overflow	Corr. sat.
Constant Residual-Mu	899.8	0	0.0000158	0.002582	0
EKF	899.8	0	0.0000158	0.002559	0
Hybrid Residual-B	899.9	0	0.0000158	0.002536	0
RLS Residual-B	899.8	0	0.0000158	0.002538	0
UKF	899.8	0	0.0000158	0.002571	0

Table 25: Software runtime-discipline counters averaged across scenario-level comparison rows. The table reports software-in-the-loop commit counts, slow-update violation rate, fast-cycle violation rate, overflow rate and correction-saturation rate. It supports software observability of the stage-and-commit contract only; it is not board commit latency, hardware reliability or FPGA timing evidence.

Mode	Avg. LER	Δ vs UKF	Δ vs full hybrid
`ukf`	0.817382	0.000000	+0.018837
`hybrid_full`	0.798545	-0.018837	0.000000
`hybrid_no_hist_deltas`	0.826723	+0.009341	+0.028178
`hybrid_no_teacher_prediction`	0.807251	-0.010131	+0.008706
`hybrid_no_teacher_params`	**0.749621**	-0.067761	-0.048924
`hybrid_no_teacher_deltas`	0.800329	-0.017053	+0.001784

Table 26: Scoped feature and teacher ablation. Negative deltas indicate lower LER than the reference. These rows support feature sensitivity, not causal teacher necessity.

Seed	Gated v5 – Full	I1 – Gated v5	Verdict
20260425	0.000907	0.163289	harmful
20260427	-0.145352	0.287166	harmful
20260428	-0.078998	0.057395	harmful
20260429	-0.127948	0.322245	harmful
20260430	-0.170777	-0.024372	mixed/no clear effect
20260510	-0.003953	-0.035533	helpful

Table 27: Descriptive six-seed mechanism and intervention summary. The lower-clip intervention does not provide causal closure and should not be described as a mitigation.

Scenario	UKF	Hybrid-b	StatCalib extension (not matched)
`static_bias_theta`	0.825370	0.810902	0.431708
`linear_ramp`	0.811201	0.787755	0.467083
`step_sigma_theta`	0.811548	0.788800	0.460016
`periodic_drift`	0.821558	0.806392	0.438751

Table 28: Supplementary statistical-calibration analysis. The table documents a separate estimator-protocol observation. Because candidate generation and selection are not matched to the main ranking protocol, these rows support the affine contract but do not establish a mature comparator or unique clean threshold. They are not a fair main-table ranking because candidate generation and selection were not matched to the main protocol.

Hardware-facing item	Current manuscript status	Required future evidence
Board platform and host	*future validation: board not connected*	Linux + FPGA host, device path, permissions, board model and driver version.
Bitstream / RTL / DMA interface	*future validation: measurement not yet run*	Bitstream hash, AXI map, DMA histogram shape, element width and timeout policy.
Fast-path latency	*future validation: measurement not yet run*	p50/p95/p99 and worst-case cycles for $Ks + b$, clipping and bank selection.
Commit latency and reliability	*future validation: measurement not yet run*	stage, commit, acknowledgement, rollback and stale-parameter counters.
Resource and power	*future validation: not measured*	LUT/FF/DSP/BRAM, clock frequency, timing closure and power.
Source-vs-board agreement	*future validation: not measured*	fixed-point agreement between software reference and board output on shared test vectors.

Table 29: Hardware-measurement requirements. These entries are requirements for future validation, not current results.

Advantage dimension	Supported basis in this study	Relation to prior work	Required upgrade
Latency-critical role	The per-shot rule is a 2×2 affine map plus clipping and bank selection; the learned estimator is outside the per-shot path.	Complements AI pre-decoders and calibration-conditioned learned decoders by placing learning in a slow calibration loop rather than in the correction datapath.	Measured board latency distribution.
Noise adaptation	The hybrid branch ranks lowest by mean `final_ler_mean` in the four tested drift scenarios; controlled stress tests expose random-walk, burst/reset and faster-than-window regimes.	Addresses a physical-layer GKP correction surface rather than only outer-code graph weights, decoder priors or message-passing reliabilities.	Repeat-expanded holdout benchmark.
Cost and verifiability	The affine count is 4 multiplications, 4 additions and 6 stored state scalars per shot, with Q4.20 software parity under controlled samples.	Gives a smaller source-vs-board verification target than a branch-aware posterior rule or a per-shot neural decoder.	Synthesis, resource and power reports.
GKP metric interpretation	`final_ler_mean` counts q/p half-lattice boundary crossings, and the residual-boundary surrogate decomposes the same events into channel-language categories.	Connects the protocol to approximate-GKP logical-boundary reasoning while remaining narrower than finite-energy channel fidelity.	Logical-channel reconstruction.
Hardware extension path	The manuscript specifies timing, commit, resource, power and source-vs-board measurements needed for an FPGA demonstration.	Uses real-time decoder literature as the measurement standard, not as evidence for this implementation.	Device logs, bitstream/RTL hash and board vectors.

Table 30: Supported comparative advantage and the evidence needed to strengthen each claim. The table summarizes what is established by the present software and analytical evidence, how that differs from adjacent GKP, learned-decoder and real-time-decoder work, and what would be required before making stronger hardware or channel-level claims.

Material class	Available for this manuscript	Interpretation limit
Predeclared simulation results	Four-scenario, five-mode ranking used in Table 11.	Supports relative ranking only inside the predeclared simulation protocol.
Ablation and mechanism materials	Feature and cross-seed intervention tables used in Tables 26 and 27.	Descriptive mechanism evidence; not a causal proof.
Statistical-calibration extension	Supplement-side comparison in Table 28.	Supports the affine-calibration contract; does not promote statistical calibration as a mature main comparator.
Controlled oracle and wrapped-Gaussian checks	One-step and short-sequence local-Gaussian comparisons in Tables 16 and 20.	Supports estimator and decoder-limit sanity checks only; does not replace formal stronger baselines or `final_ler_mean` validation.
Holdout drift stress tests	Random-walk, burst/reset and faster-than-window controlled stress families in Table 21.	Supports non-hardware unseen-drift diagnostics only; does not establish trained-branch holdout generalization, confidence intervals or hardware robustness.
Residual-boundary channel surrogate	q/p half-lattice crossing event decomposition in Table 17 and method-level lattice sanity summary in Table 19.	Supports a Pauli-event surrogate bridge between residual-boundary scoring and channel language only; does not estimate finite-energy logical-channel fidelity, process fidelity or hardware fidelity.
Fast-path cost model	Analytical operation-count comparison in Table 23.	Supports low-complexity motivation only; does not establish FPGA timing, power or resource use.
Fixed-point parity check	Q4.20 software-emulation comparison in Table 24.	Supports numerical fixed-point feasibility under controlled samples only; does not establish source-vs-board agreement, synthesis, timing closure, resource use or power.
Literature metric crosswalk	Cited metric anchors for the external comparison table, including local literature-card figure/table/equation anchors or explicit card-level limitations.	Supports related-work positioning only; not a normalized leaderboard and not source data for the present experiments.
Training and runtime support	A clean CPU-only rerun and selected isolated `.tflite` runtime checks.	Does not establish full reproducibility, default-environment portability or deployment closure.
Hardware validation requirements	Validation requirements without current board measurements.	Does not establish board execution, timing, resource or source-vs-board agreement.

Table 31: Availability of supporting materials. Each row states what the available materials can support without upgrading the claim level.

Evidence layer	Supported statement	Not established by the present study
Predeclared simulation benchmark	Four-scenario, five-mode ranking under the predeclared protocol.	Expanded benchmark, `.tflite` ranking or board ranking.
Ablation/mechanism	Descriptive feature sensitivity and cross-seed mechanism material.	Causal mechanism proof or intervention success.
Training/material	Canonical material chain and one clean CPU-only rerun.	Full reproducibility, cross-host portability or GPU/Linux closure.
Isolated runtime	Selected preserved `.tflite` artifacts execute in one local software environment.	Default-environment compatibility, HIL closure or deployment closure.
Hardware validation	No board-level measurement is reported in this study.	Real-board execution success or hardware validation.

Table 32: Validation-scope table for manuscript claims.

Result class	Source data	Protocol anchor	Permitted interpretation
Main simulation benchmark	Machine-readable comparison table and run summary.	Predeclared four-scenario, five-mode, paired-seed, repeats = 2 protocol.	Use for the predeclared ranking only; do not upgrade to expanded benchmark, `.tflite`, real-board or paper-grade SOTA evidence.
Feature and teacher ablation	Ablation summary table and source-data file.	Same simulation setting, with controlled feature or teacher inputs removed.	Use for feature-sensitivity wording; do not claim teacher-parameter necessity or causal closure.
Multi-seed mechanism summary	Six-seed intervention summary and figure source data.	Matched descriptive intervention comparison across the listed seeds.	Use as descriptive mechanism evidence; do not claim intervention success or a closed causal mechanism.
Supplementary statistical calibration	Statistical-calibration comparison rows and source-data file.	Separate supplementary estimator protocol.	Use as supplement-side evidence that the affine contract admits a non-neural estimator; do not fold it into the main ranked table.
Hardware validation requirements	No board timing/resource rows are reported; selected `.tflite` checks are used only as software-runtime support.	Future board validation must provide device logs, bitstream or firmware hashes, DMA evidence and source-vs-board vectors.	Use only to define future measurement requirements; do not report board execution, deployment closure or source-vs-board agreement.

Table 33: Benchmark protocol and source-data map. The table records the source-data route used by the present figures and supplementary files.

Theoretical object	Current implementation bridge	Current manuscript use	Missing before stronger claim
Syndrome stream s_t	Simulation sessions generate recent syndrome statistics from configured synthetic drift trajectories.	Defines the fast/slow-loop interface and the histogram-window inputs.	Physics-level syndrome wrapping and logical-channel validation.
Affine fast path	Runtime contract applies a clipped and quantized $\Delta_t = K_t s_t + b_t$.	Core method claim: complex estimation is kept outside the fast path.	Bit-accurate fixed-point and board-latency evidence.
Effective drift state	Four predeclared synthetic drift families instantiate static, ramp, step and periodic perturbations of $(\sigma, \mu_q, \mu_p, \vartheta)$.	Controlled synthetic probes of adaptation demands.	Machine-readable field-to-symbol mapping and holdout drift families.
Main metric	The comparison table reports `final_ler_mean` means and descriptive standard deviations for paired seeds with repeats = 2.	Predeclared simulation ranking.	Confidence intervals, larger repeat count and stronger logical-error validation.
Hardware target	Board-facing entries specify future measurement requirements.	Future board-validation protocol only.	Device logs, bitstream or RTL hashes, DMA evidence and timing/resource data.

Table 34: Theory-to-configuration bridge for the manuscript. The bridge records how the notation in the method sections is instantiated by the predeclared simulation protocol, while making clear that the current data are not a physical-noise or hardware-validation result.

Layer	Artifact or runtime class	Current manuscript use	Not established by the present study
Main comparison	software-in-the-loop simulation rows from `comparison.csv` and `summary.json`.	Predeclared four-scenario, five-mode ranking.	True `.tflite`, real-board or expanded benchmark ranking.
Hybrid residual branch	Preserved `.npz` model artifact recorded in the hybrid rows.	Slow-loop model-artifact evidence for the current hybrid software-in-the-loop simulation branch.	Portable deployment or board execution.
Baseline modes	software-in-the-loop simulation modes with no model artifact path in the rows.	Classical and residual baselines under the same protocol.	Learned artifact parity or `.tflite` baseline evidence.
Software runtime check	Selected preserved `.tflite` files executed in one local software environment; JSON metadata artifacts are not treated as executables.	Supporting runtime evidence only.	Default-environment compatibility, HIL closure or deployment closure.
Non-executable metadata artifacts	JSON-sidecar or metadata-only artifacts in the runtime abstraction.	Artifact taxonomy only.	True TFLite execution.
Hardware validation requirements	Validation requirements without a current board measurement.	Future hardware measurement requirements.	Real-board execution success or hardware validation.
Statistical calibration	Supplementary statistical-calibration source data.	Evidence that the affine contract can host a non-neural estimator.	Mature main-comparator status or replacement of the main table.

Table 35: Runtime-artifact validation scope. The table separates simulation results, model artifacts, true `.tflite` support, non-executable metadata artifacts, hardware-validation requirements and supplementary statistical-calibration evidence so that the manuscript does not upgrade one artifact class into another.

Coverage group	Mechanically checked surface	Interpretation limit
Main performance tables	Main results, paired deltas, paired-uncertainty and LER advantage-margin source rows are checked against source CSV files.	Descriptive two-repeat software comparison only; not confidence intervals, standard errors, p-values, significance tests or an expanded benchmark.
Controlled diagnostics	Oracle-affine, logical-surrogate, surrogate-fidelity, lattice-channel sanity, boundary-sensitivity, sequence-baseline, holdout-stress and commit-lag tables are checked against generated CSV files.	Diagnostic layers only; not trained-branch holdout proof, finite-energy logical-channel fidelity or hardware latency.
Implementation feasibility	Fast-path cost, fixed-point parity, runtime discipline and validation-contract source rows are checked against generated or figure-source CSV files.	Analytical/software feasibility only; not FPGA synthesis, timing, resource, power or source-vs-board evidence.
Source and literature maps	Metric-readiness, literature crosswalk, row provenance, figure manifest and file-level source manifest are checked for row, citation or hash consistency.	Not a leaderboard, not recursive historical-run hash closure and not full reproducibility proof.
Planning and boundary surfaces	Benchmark-expansion, runner-smoke, availability, validation-scope, runtime-artifact and hardware-plan surfaces are classified separately.	Planning or requirements surfaces only; not current result evidence, hardware validation or submission-completeness proof.

Table 36: Source-data coverage matrix. The matrix identifies which manuscript table families are mechanically checked and which remain planning or boundary surfaces. It is a traceability aid, not an evidence upgrade.

Layer	Current evidence	Manuscript use	Missing for stronger claim
Main comparison	Predeclared four-scenario, five-mode software-in-the-loop simulation benchmark with paired seeds, repeats = 2, paired-delta and resampling-envelope tables.	Predeclared-set ranking; hybrid residual branch has the lowest mean `final_ler_mean` in all four scenarios; paired tables provide source-data transparency only.	Paired CI, more repeats and a formal expanded holdout benchmark.
Error bars	Figure source data contain mean plus descriptive SD for Fig. 2.	Visual uncertainty marker only.	Do not treat as CI, standard error or statistical significance.
Source data	`comparison.csv`, `summary.json`, figure source CSVs, figure manifest, source-data consistency report and file-level source-data manifest plus row-level provenance manifest for the main software-in-the-loop simulation rows.	Table and figure source-data support for the manuscript.	Whole-manuscript source-data check, recursive historical run hash closure and independent reproduction.
Reproducibility	Recorded run metadata, host checks and scoped training/material support.	Available repository-artifact reproducibility.	Repeated runs, cross-host checks or containerized reproduction.
Hardware path	No current board measurement is available.	Future measurement surface only.	Device logs, bitstream/RTL hash, DMA evidence, latency and resource data, and source-to-board vectors.

Table 37: Statistical treatment and reproducibility. The current data support a scoped software-in-the-loop simulation ranking with descriptive uncertainty; stronger statistical, reproducibility or hardware claims require the missing evidence listed in the rightmost column.

Manuscript claim	Current evidence	Required source-data treatment
Drift adaptation can be expressed as a runtime affine calibration problem.	Problem formulation, architecture contract and predeclared software-in-the-loop simulation protocol.	Keep as method claim; source data are definitions, protocol metadata and parameter-flow schematic, not performance measurements.
The main-table hybrid residual branch has the lowest mean `final_ler_mean` in the predeclared four-scenario set.	Table 11 from the predeclared software-in-the-loop simulation benchmark.	Archive machine-readable per-scenario rows; add seed-level uncertainty before claiming statistical robustness.
Feature choices affect residual calibration behavior.	Table 26; scoped feature and teacher ablations.	Treat as an appendix/source-data table; do not convert it into a teacher-necessity claim.
The mechanism story remains descriptive.	Table 27; six-seed intervention summary with mixed or harmful intervention outcomes.	Link the figure source data and trace summaries; state that causal closure is missing.

Table 38: Primary claim scope and source-data map for the manuscript. The table records the source-data treatment required for method, main-result and mechanism claims.

Manuscript claim	Current evidence	Required source-data treatment
The metric set is intentionally layered.	Table 4; `final_ler_mean`, controlled checks, the residual-boundary Pauli-event surrogate, cost counts and fixed-point software parity are current metrics, while finite-energy channel fidelity and hardware timing remain external standards.	Keep the source CSV with the manuscript; do not relabel `final_ler_mean` or the surrogate average-fidelity field as channel fidelity or measured latency.
Controlled oracle-affine and wrapped-Gaussian checks are diagnostic baselines.	Tables 16 and 20; controlled local-Gaussian source CSVs.	Keep the one-step and short-sequence protocols separate from the predeclared software-in-the-loop simulation ranking; do not write them as a full nearest-lattice, known-noise or wrapped-decoder benchmark.
Holdout drift stress tests probe adaptation space, not trained-branch generalization.	Table 21; controlled random-walk, burst/reset and faster-than-window diagnostic source CSV.	Keep as non-hardware stress diagnostics; do not relabel the rows as expanded benchmark evidence, confidence intervals or robustness proof.
Runtime counters and the validation-contract figure support software observability.	Table 25 and Fig. 3; runtime-discipline and validation-contract source CSVs.	Keep as software protocol observability and source-data summary; do not write these rows as board latency, hardware reliability, synthesis, timing closure or source-vs-board agreement.
The fixed-point claim is numerical, not hardware-complete.	Table 24; software-emulation source CSV; Q4.20 emulation introduces no residual-boundary crossing change in the controlled sample set.	Do not write this as source-vs-board agreement, timing closure, resource measurement, power measurement or real-board latency.
The affine contract can host non-neural calibration.	Table 28; supplementary statistical-calibration analysis.	Keep in supplement-side source data with no-main-comparator and no-unique threshold notes.
Hardware validation is a future surface.	Table 29 and the absence of board measurements in this study.	Retain these requirements until board logs, bitstream hashes, latency/resource data and source-vs-board vectors exist.

Table 39: Diagnostic, runtime and validation claim scope for the manuscript. The table continues the source-data map for controlled diagnostics, numerical checks, supplementary calibration and hardware-facing requirements.

Validation concern	Present manuscript treatment	Evidence still needed for a stronger claim
The benchmark is too narrow.	The claim is restricted to the predeclared four-scenario, five-mode software-in-the-loop simulation protocol, and the protocol is reported as a predeclared simulation benchmark.	A predeclared expanded benchmark with holdout drift families, stronger baselines and more repeats or uncertainty estimates.
The method lacks strong theoretical baselines.	The text adds controlled one-step and short-sequence oracle-affine / wrapped-Gaussian checks and does not claim superiority over nearest-lattice or fully tuned sequence-level wrapped decoders.	A formal known-noise affine, nearest-lattice or wrapped-Gaussian baseline under a predeclared benchmark protocol.
The mechanism is not causal.	Ablation and intervention materials are described as feature sensitivity and descriptive mechanism evidence only.	Mechanism-specific perturbations, trace-level causal isolation or a predictive failure-mode test.
Hardware claims require direct measurements.	Board-level validation is stated as a future measurement surface because no board measurements are reported here.	Device logs, bitstream hashes, fixed-point agreement, latency/resource measurements and source-vs-board tests.
The neural branch is not the only plausible estimator.	Statistical calibration is kept as a supplementary analysis that supports the affine contract rather than a main comparator.	A future predeclared comparator selection protocol if the statistical estimator is to be used in the main ranking.
The work may not be reproducible outside the present software environment.	Reproducibility statements are restricted to available repository artifacts and single-host checks.	Repeated-run, cross-host or containerized reproduction with explicit dependency locks and artifact manifests.

Table 40: Limitations and required validation for stronger claims. Each row states how the manuscript limits the present claim and what evidence would be required before stronger wording is justified.

Term	Meaning in this manuscript	Do not imply
Runtime-consistent affine calibration	A slow loop updates (K_t, b_t), while the fast path applies $\Delta_t = K_t s_t + b_t$.	Full posterior decoding or hardware closure.
Fast path	The per-shot deterministic affine correction surface.	A measured FPGA implementation, unless future board evidence is added.
Slow calibration loop	A teacher, neural residual branch or statistical estimator that updates affine parameters from recent syndrome statistics.	A requirement that the slow loop must be neural.
Teacher-anchored hybrid residual branch	The predeclared main-table hybrid mode; ablation rows do not establish teacher-parameter necessity.	A universal SOTA decoder or causal mechanism proof.
Supplementary statistical calibration	A supplement-side check that non-neural estimators can also satisfy the affine contract.	Main-comparator status.
software-in-the-loop simulation	A source-controlled software protocol for testing the runtime calibration interface under synthetic drift.	Real-board HIL or deployment success.
Hardware validation check	A record that no current board measurement is available and that future measurements must provide timing, resource and source-vs-board data.	Successful board execution.

Table 41: Reader-facing terminology. This table prevents support-layer terms from being misread as stronger experimental claims.

Figure	Main conclusion	Required source data	Current status
Fig. 1 architecture	GKP drift adaptation is expressed as a dual-loop affine runtime contract.	Method schematic, parameter-flow definition and validation scope notes.	Python manuscript figure with schematic source map.
Fig. 2 main software-in-the-loop simulation results	Hybrid residual calibration is best in the predeclared software-in-the-loop simulation table.	Main comparison source table, `final_ler_mean` mean, descriptive SD and later paired comparison.	Python manuscript figure with mean + descriptive SD source data; CI still missing.
Fig. 3 ablation and mechanism	Feature and histogram dynamics constrain the mechanism story.	Ablation rows, cross-seed intervention rows and trace summaries.	Python manuscript figure with descriptive-only source data.
Fig. 4 statistical calibration	Non-neural calibration can occupy the same affine contract without becoming the main comparator.	Supplementary statistical-calibration rows and no-main-comparator notes.	Python manuscript figure; supplementary analysis only.
Fig. 5 validation contract	The present validation package supports a software fast-path contract across analytical arithmetic, Q4.20 parity and runtime counter observability.	Validation-contract source table plus fast-path cost, fixed-point parity and runtime-discipline source tables.	Python manuscript figure with source-data-backed software summary only; no FPGA timing, resource, source-vs-board or board-latency measurement.

Table 42: Figure source-data status. Each figure is tied to the source data or validation record needed to support its intended conclusion.